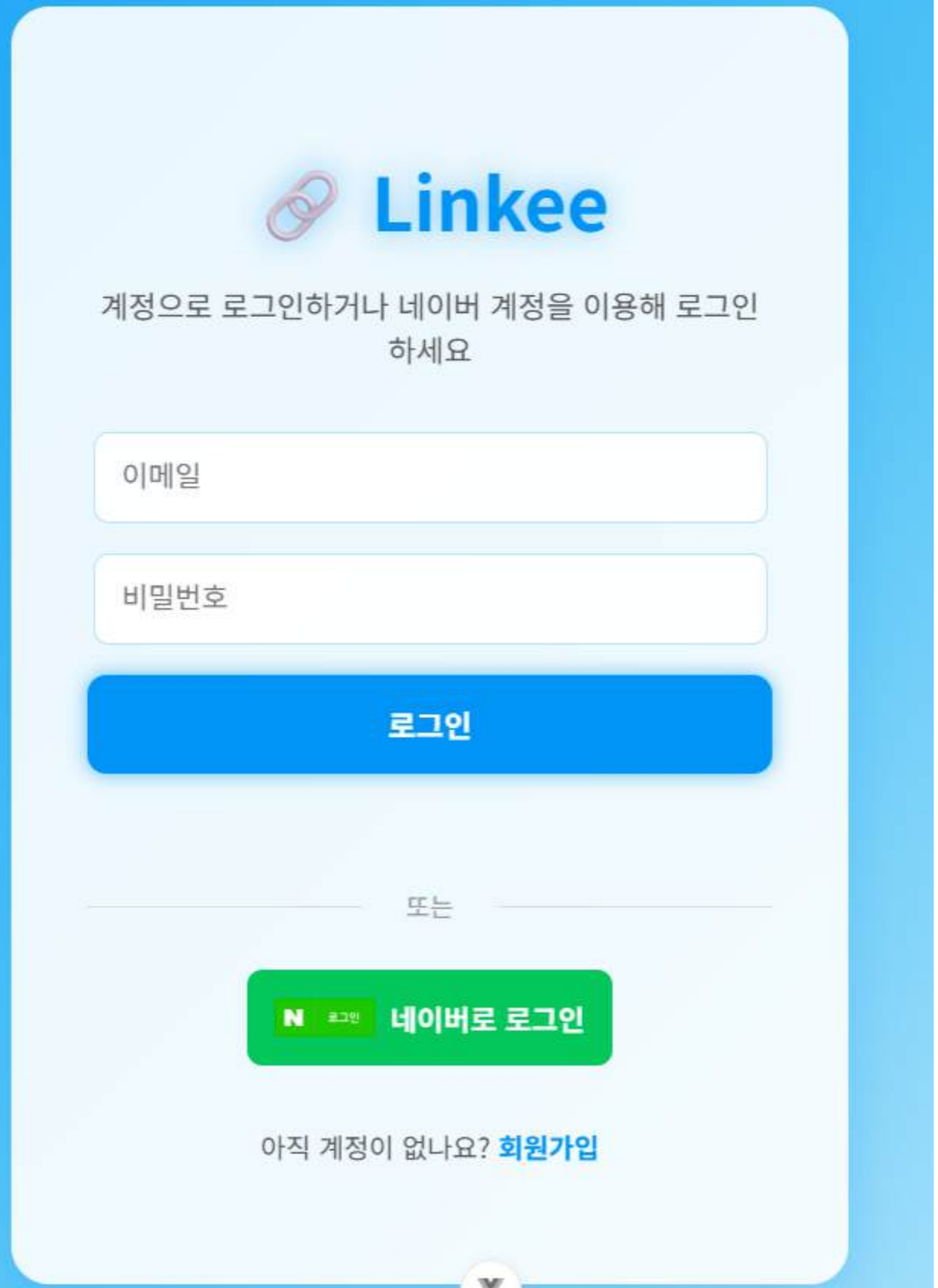


Linkee Project
Vue

team : 김상재 김진 김명진 정동욱 유한세

로그인&회원가입



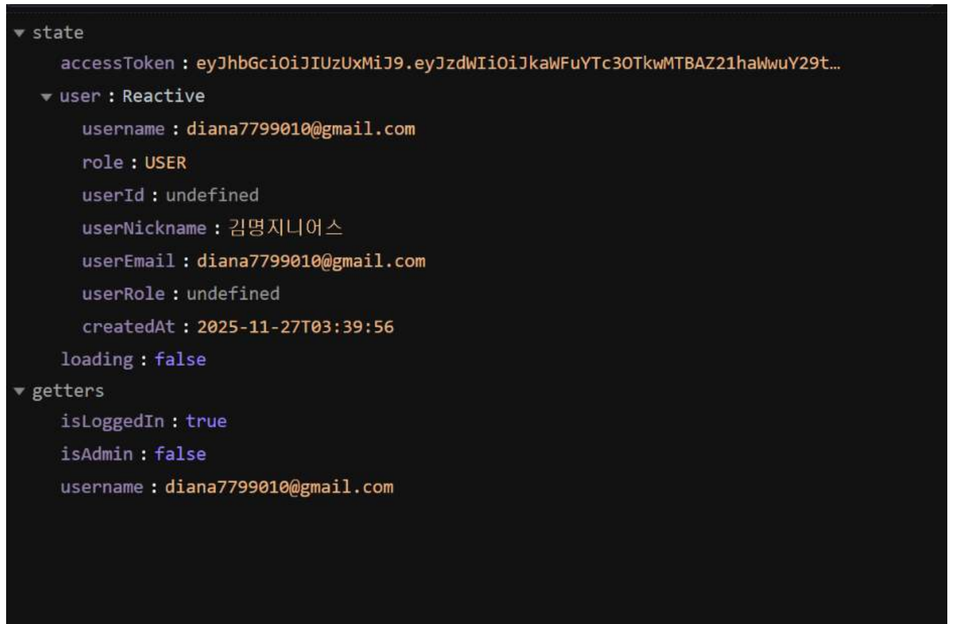
Login

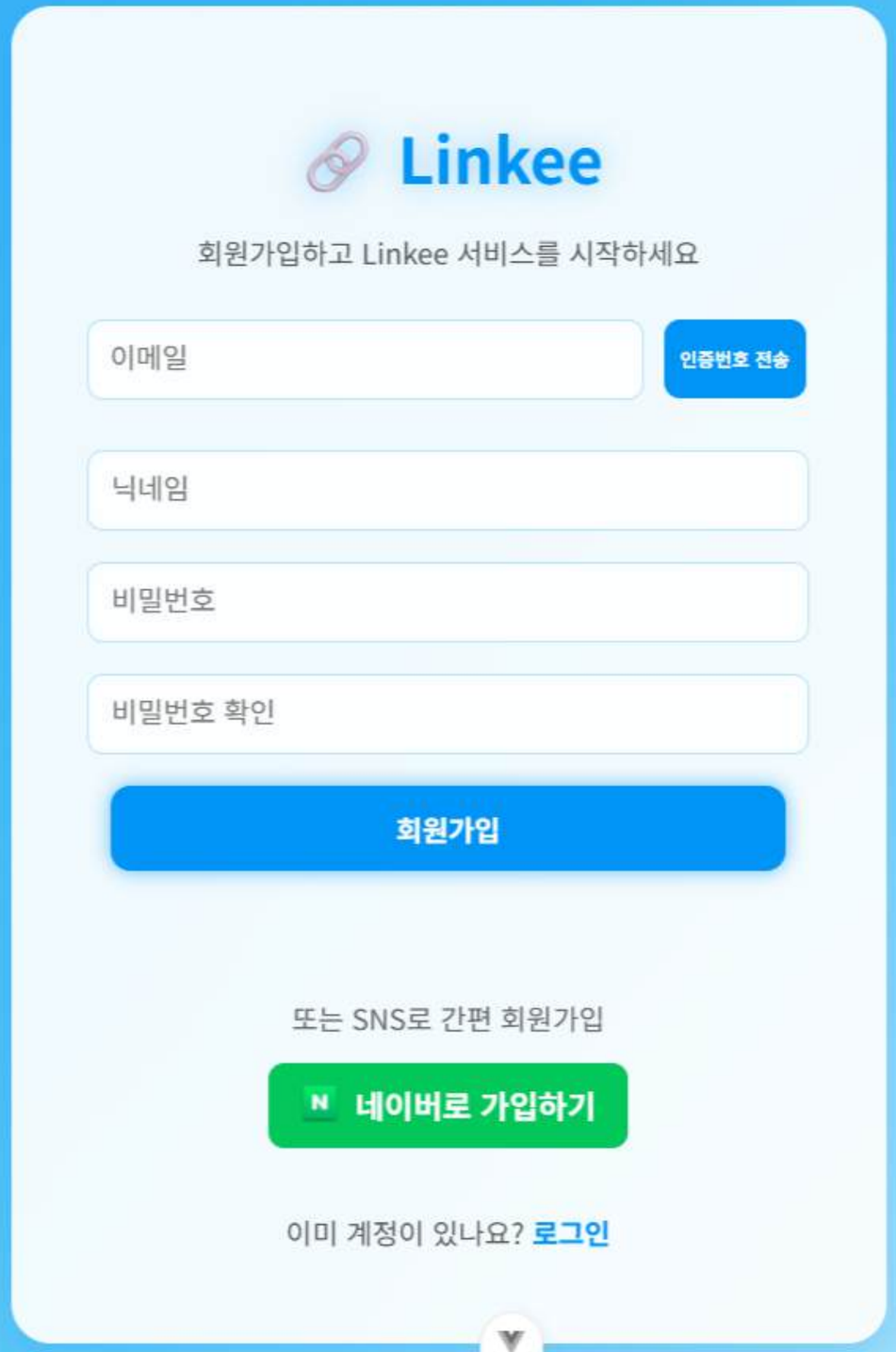
Location : '/' '/login'

일반 유저 로그인

일반 계정 로그인과 네이버로그인 연동

Pinia에 해당 정보 저장





signup

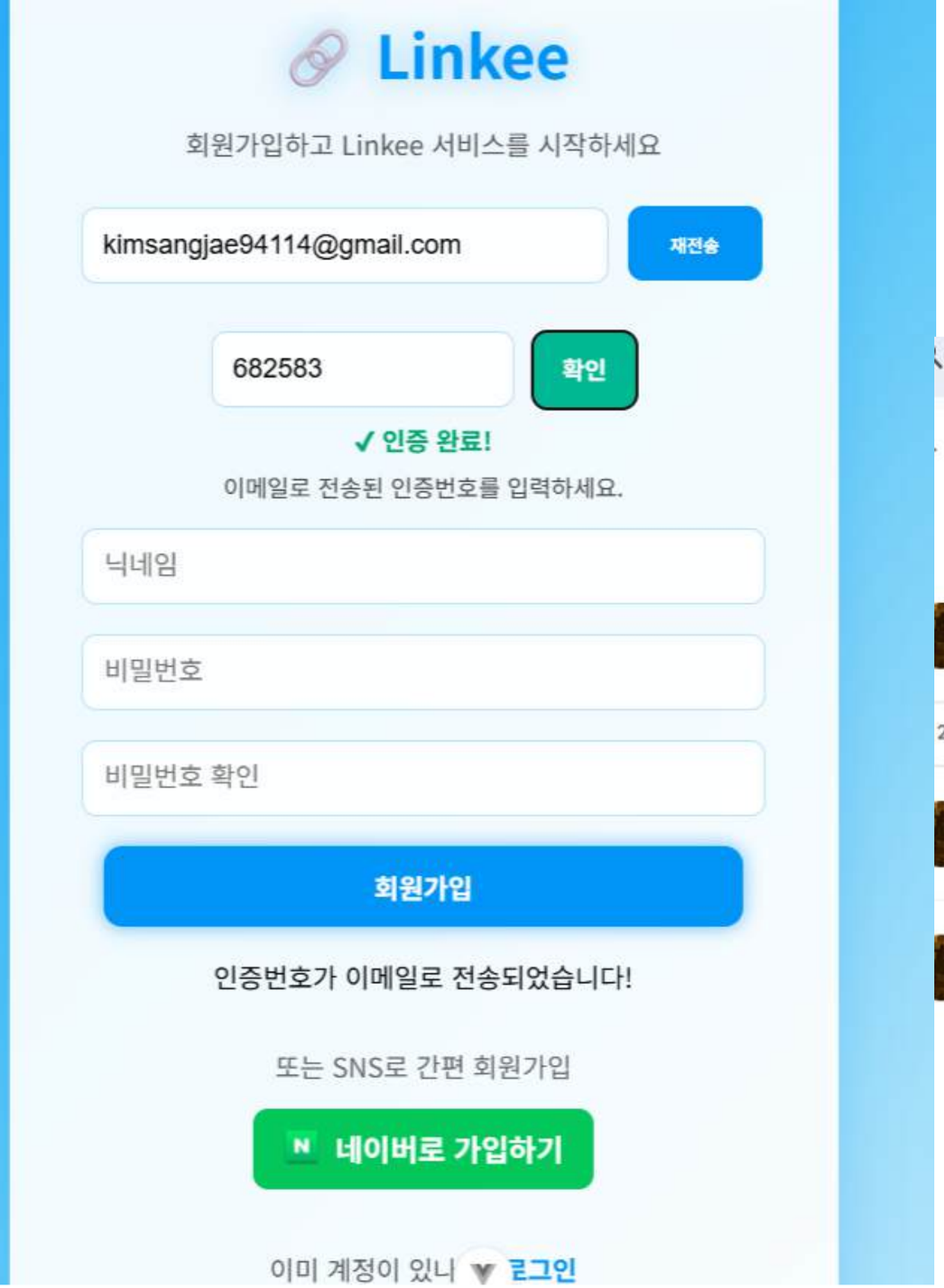
Location : '/signup'

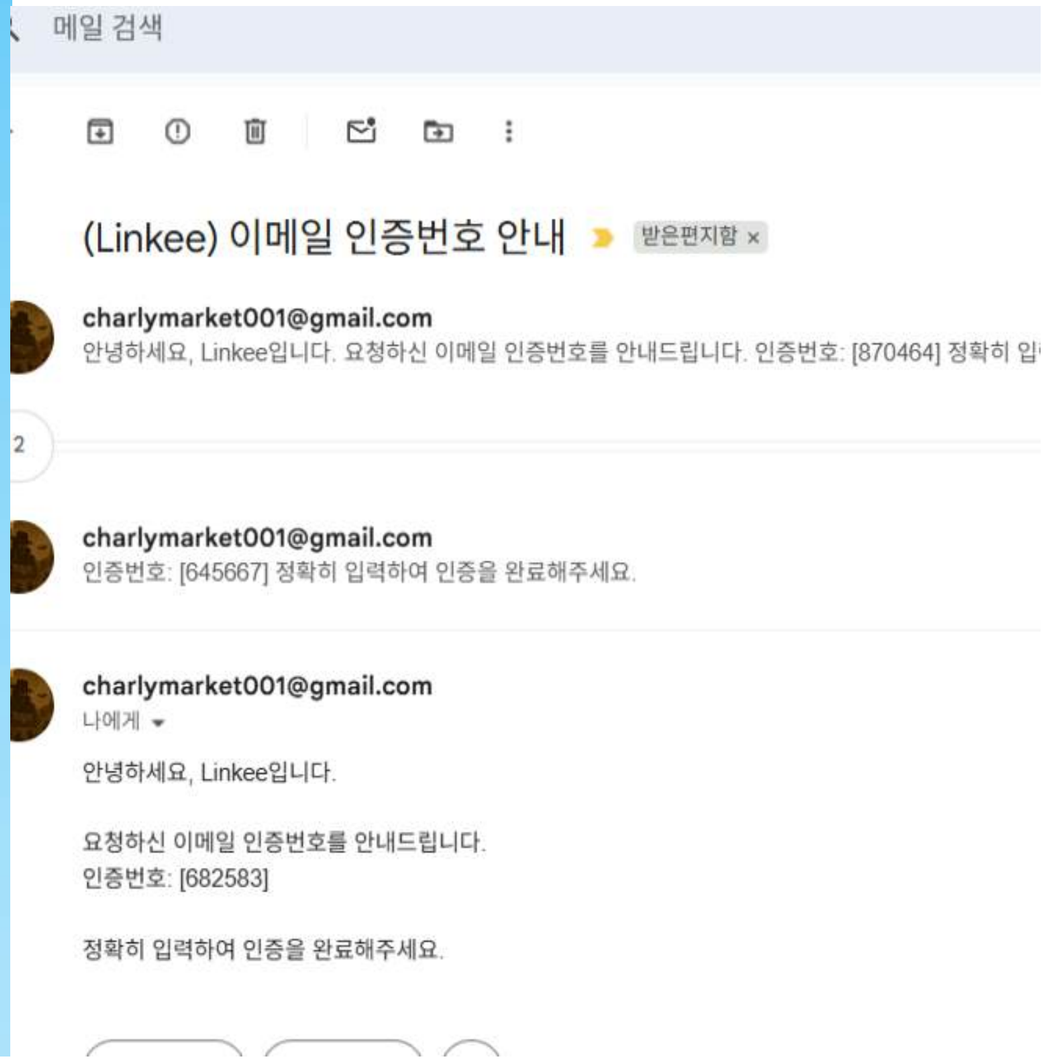
일반 유저 회원가입

이메일 인증 smtp 통해
인증 후 필수 입력값 적어서
회원가입 할 수 있다

email smtp

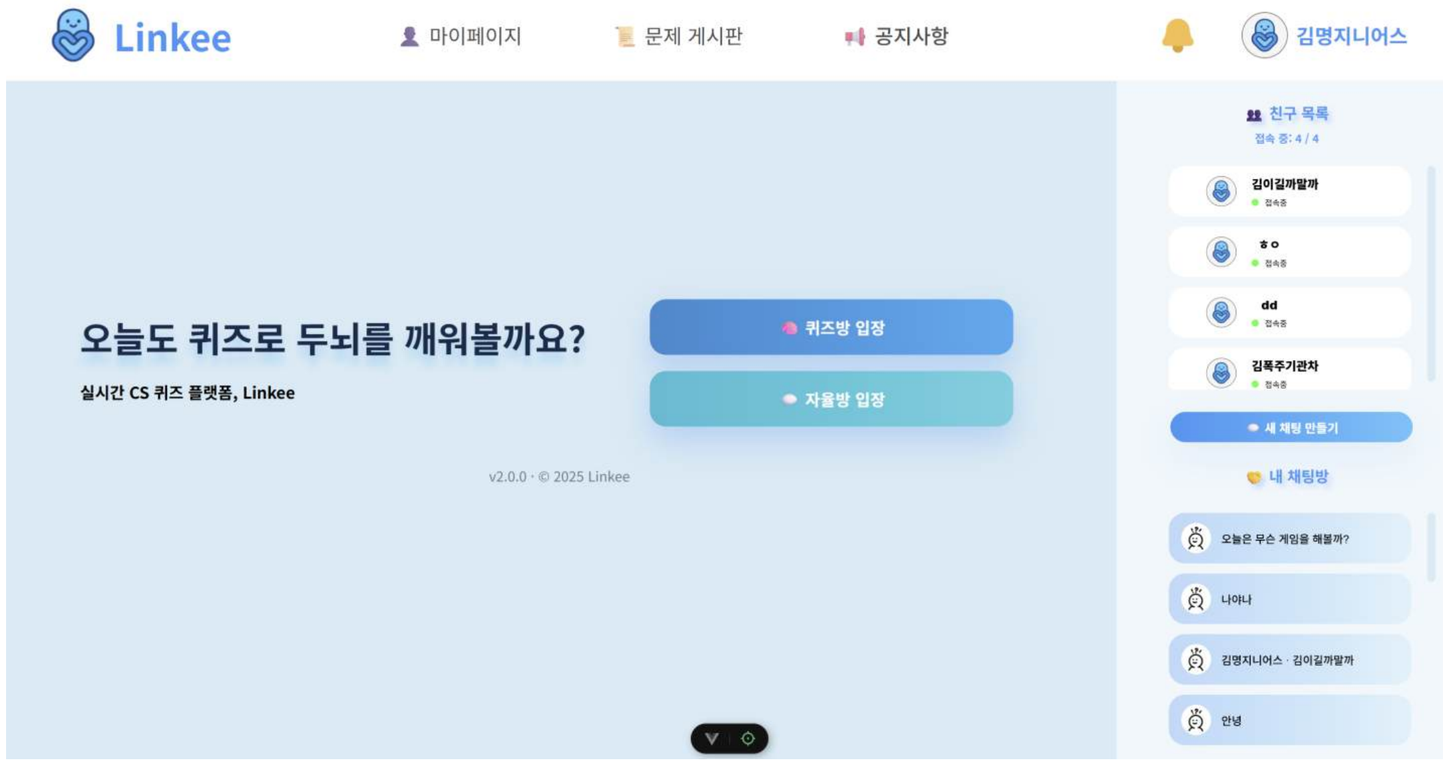
구글 smtp 이메일 인증





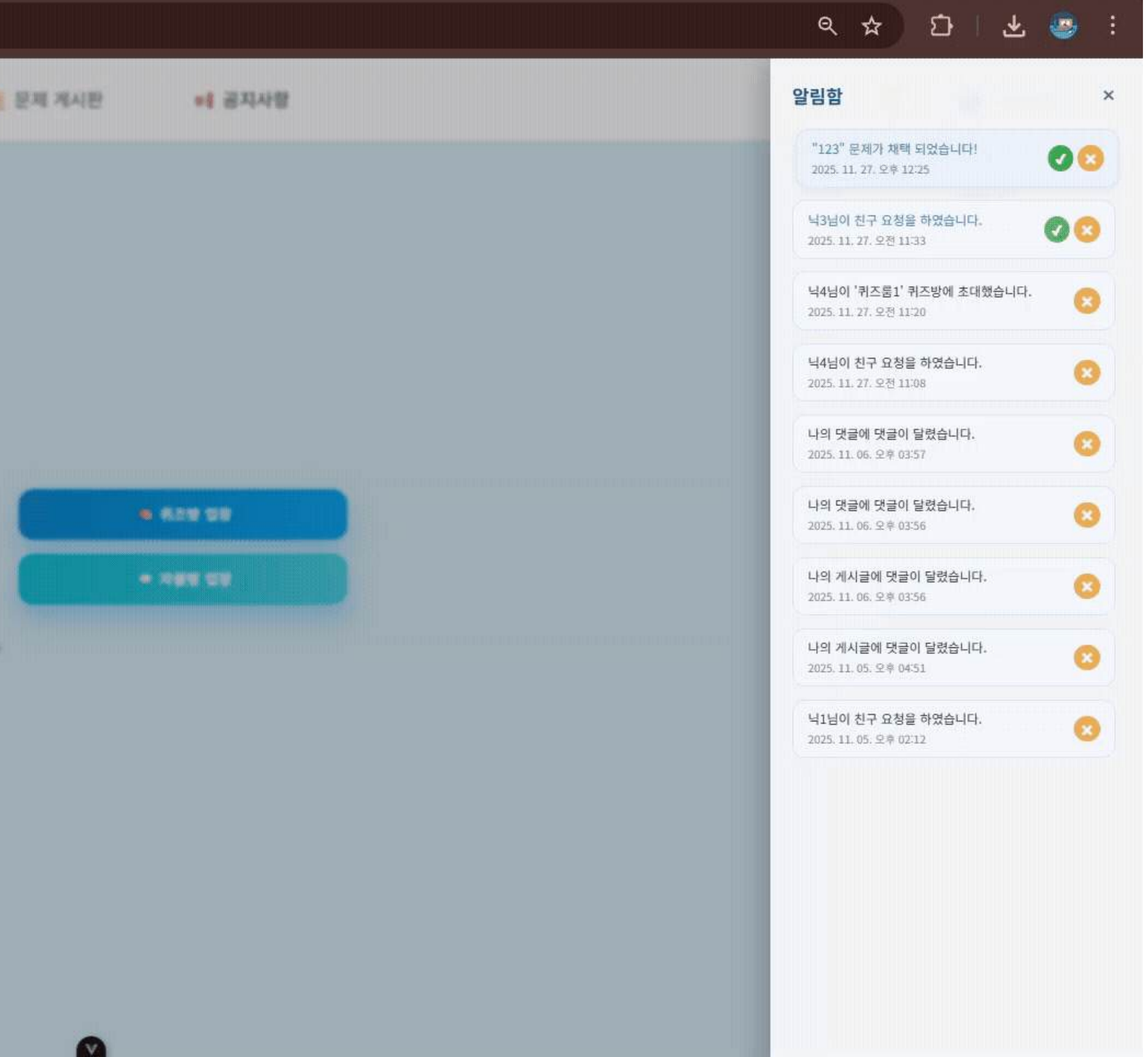
홈페이지

홈페이지



Location : '/home'

자율방, 퀴즈방으로 이동할 수 있는 홈페이지이다.
상단바와 사이드바를 통해 원하는 페이지로 이동할 수 있으며 채팅을 할 수 있다.



알람박스

알림이 발생되는 이벤트

1. 친구요청
2. 방 초대
3. 내가 올린 문의 글에 관리자의 답변
4. 내가 올린 문제 글이 검증 완료
5. 나의 게시글에 답변
6. 나의 댓글에 답변

이러한 API가 해당 유저에게 호출이되면 유저의 개인 알림박스에 쌓이며 ✔ 버튼을 누르면 읽음 처리가 되고, X 표시를 누르면 알림이 완전히 삭제가 됩니다. 이때 화면에는 팝업 로고로 안내를 합니다.





사이드바



친구목록

1. 사용 기술
VVue 3 (Composition API)
Axios
Pinia (사용자 정보 상태 관리)
반응형 렌더링 (v-for)

2. 동작 방식
1) 친구 목록 데이터 가져오기

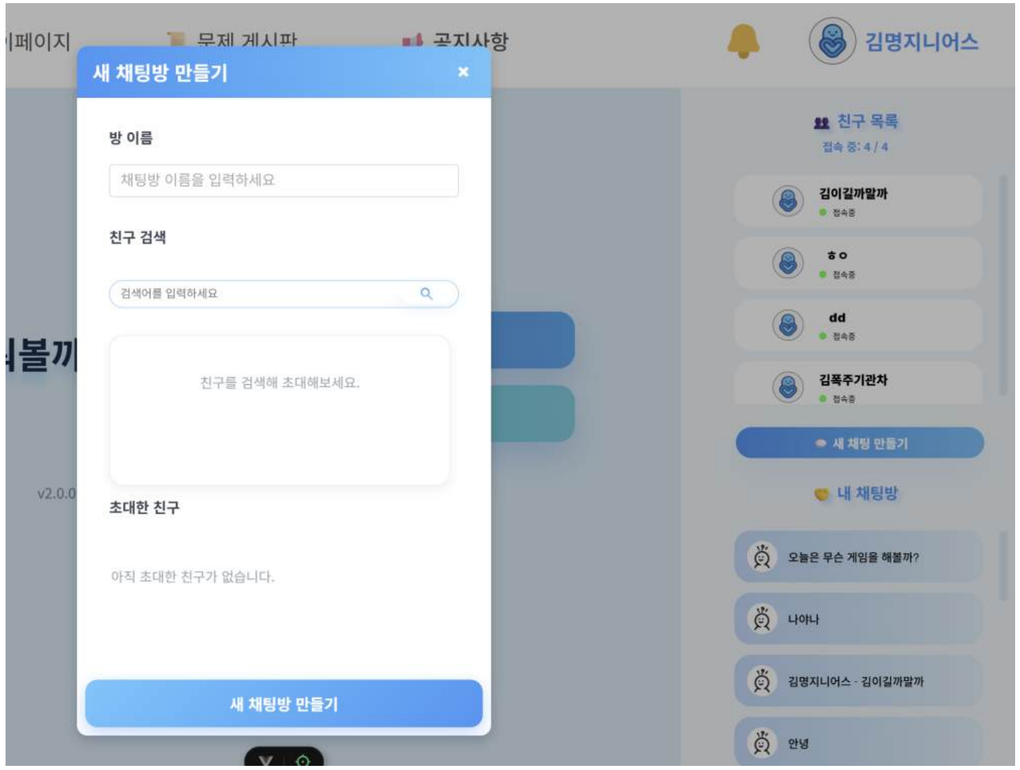
프론트에서는 먼저 API 호출 → 응답 데이터를 프론트에서 재가공하여 사용하기 좋게 정제한다.

- API 구조를 프론트에서 재정의
백엔드 응답은 친구 관계 객체 (requesterId / receiverId) 구조라 바로 UI에 쓰기 힘듦.
그래서 프론트에서 내가 요청자인지, 상대가 요청자인지 판별하여:
친구의 id
친구의 닉네임(name)
접속 상태
아바타 이미지
이런 UI 친화적인 형태로 직접 맵핑함.

2)반응형 상태로 관리

Vue Composition API에서 ref([])로 관리하여
friends 배열이 바뀌면 UI도 자동으로 업데이트됨.

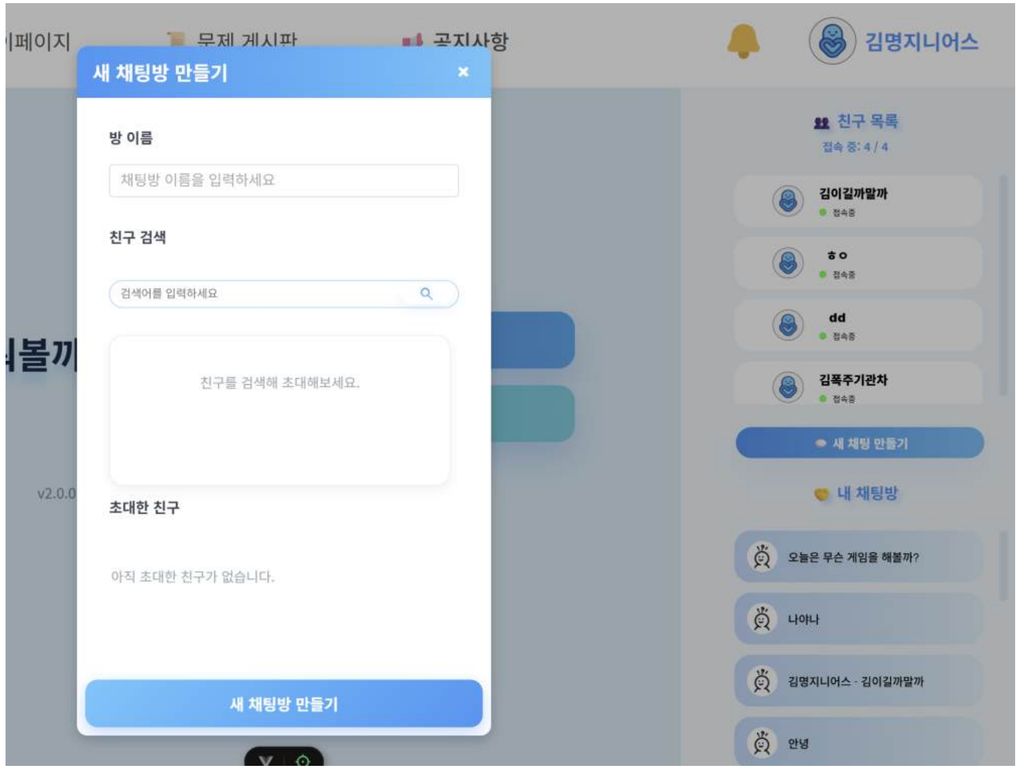
새채팅방 만들기



1. 사용 기술
Vue 모달 컴포넌트
양방향 바인딩 (v-model)
axios + JWT 자동 포함 (interceptor)

2. 동작 방식
핵심 구현 포인트
모달에서 친구 선택 → 부모 컴포넌트로 emit
데이터 구조를 프론트에서 직접 재가공하여 API로 전달
생성 후 자동으로 사이드바 채팅방 목록 업데이트

채팅 멤버 조회 기능



1. 사용 기술

Axios

Vue 반응형 배열 관리

모달 & 채팅창 컴포넌트 구조화

2. 동작 방식

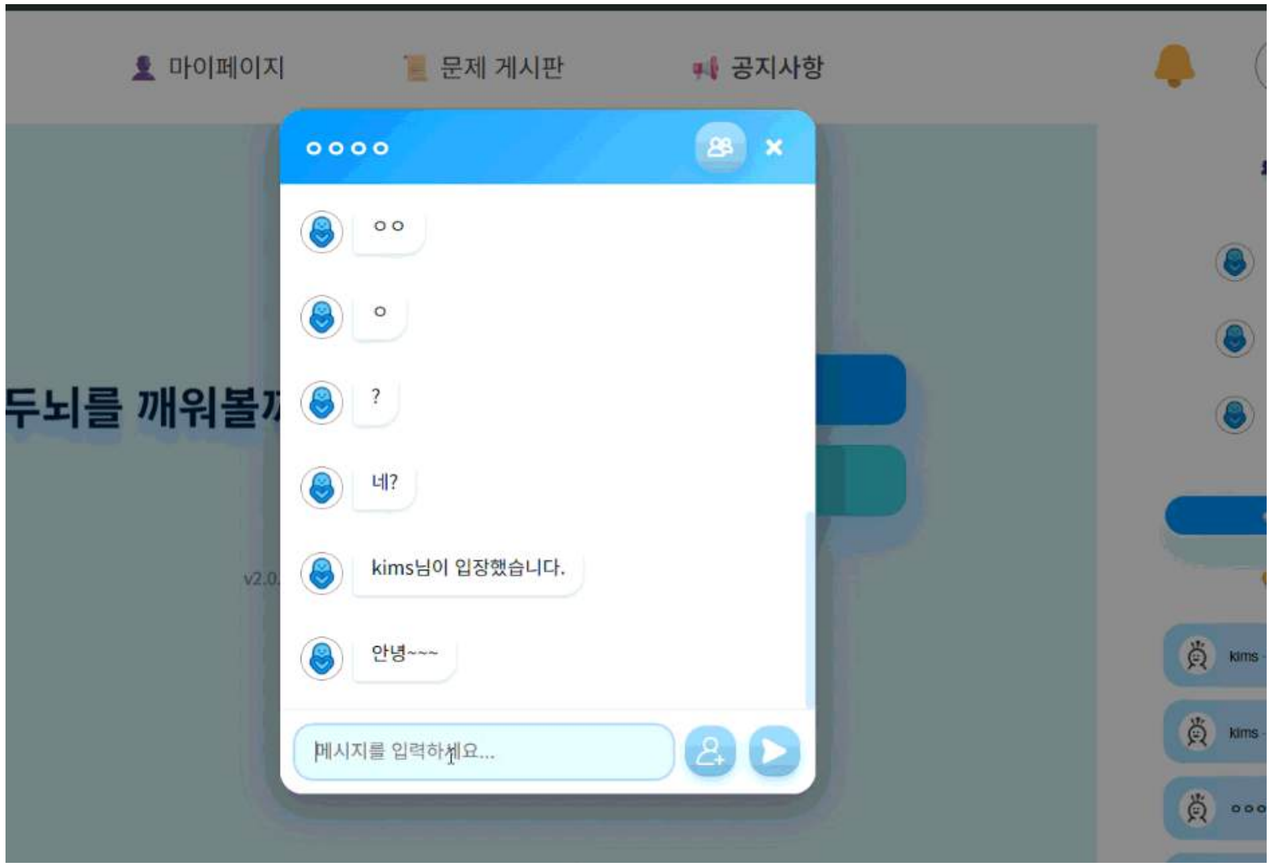
1) 방 클릭하면 모달 열기 → “입장” 누르면 멤버 조회

- 멤버 정보를 반응형 상태 (room.members)에 저장
- 전개 연산자({ ...room })로 새로운 객체 생성 → 모달에 전달 시 반응성 보장
- 멤버 조회는 입장 직전에 진행해 최신 상태 유지

2) 채팅방 멤버 UI 표시

ChattingModal.vue에서 members 배열을 받아 렌더링

채팅



1) Props + Emits 기반 독립 모달 UI
메시지 렌더링과 입력 UI만 담당
전송 로직과 상태 관리는 부모 컴포넌트에서 처리

2) 채팅방 입장 시 초기 로딩
부모 컴포넌트에서 WebSocket(STOMP) 연결
멤버 목록 및 기존 메시지 조회 후 모달에 전달

3) 실시간 메시징 처리
STOMP 구독
수신 메시지를 즉시 UI에 반영
사용자가 보낸 메시지는 emit → WebSocket publish 전송

4) 부가 기능 연동
모달 내부에서 방 멤버 보기 · 친구 초대 지원
초대 완료 후 API로 멤버 목록 재조회 → UI 자동 갱신

공지사항

공지사항



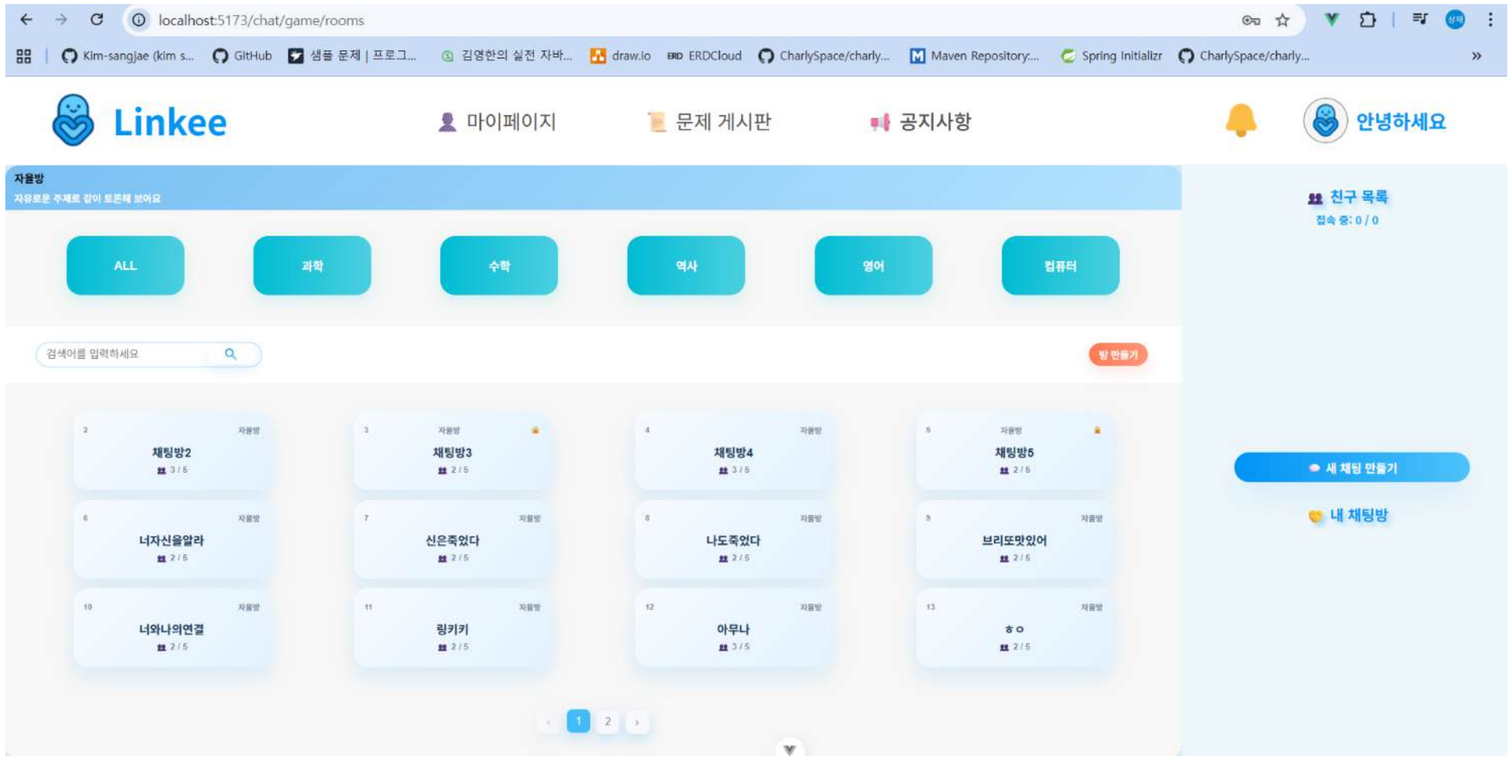
1. 사용 기술
Vue 3 (Composition API)
Axios 기반 API 모듈
Pinia (로그인 유저 상태 — optional)
v-for 렌더링
Pagination 처리(optional)

2. 동작 방식

공지사항 조회는 Axios로 /notice 목록을 가져와 Vue Composition API의 반응형 상태로 렌더링하고, 각각의 카드를 클릭하면 상세 페이지로 이동하는 구조로 구현.

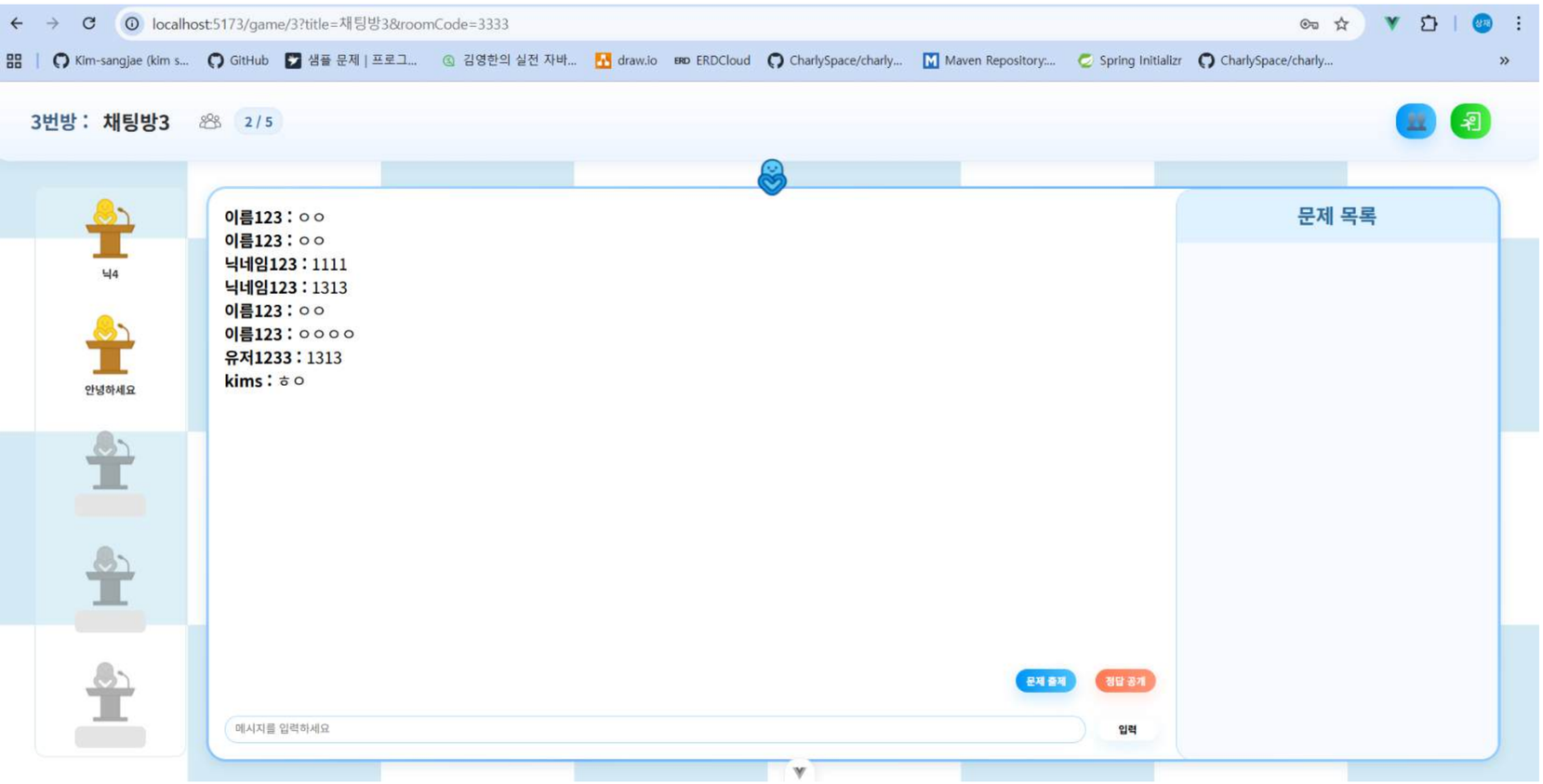
자율방

방목록



- 존재하는 게임방 목록 조회
- 카테고리 , 검색을통해 정렬조회
- 페이징을 통한 조회

자율방



목록 중 방을 선택하여
- 웹소켓 통신 연결

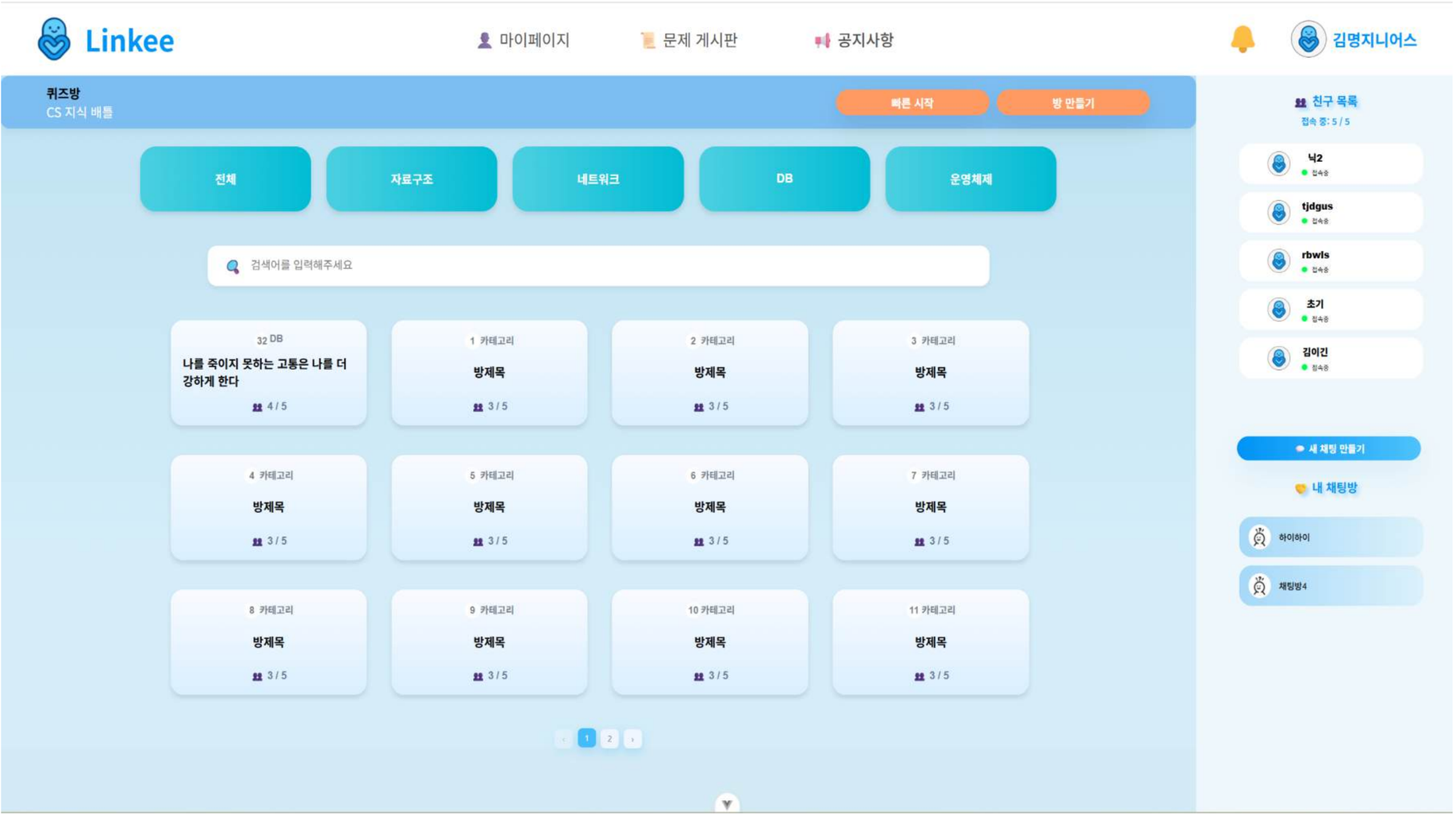
- 해당 방에 구독(subscrip)

- 실시간 stomp 메세지 통신방식으로
  메시지 전송 , mongoDB 메세지저장
  해당방의 구독자들에게 메세지 실시간 전송

- 피니아에 방의 정보를 저장하여 실시간
  업데이트를 좀 더 용이하게 반영

퀴즈방

quiz



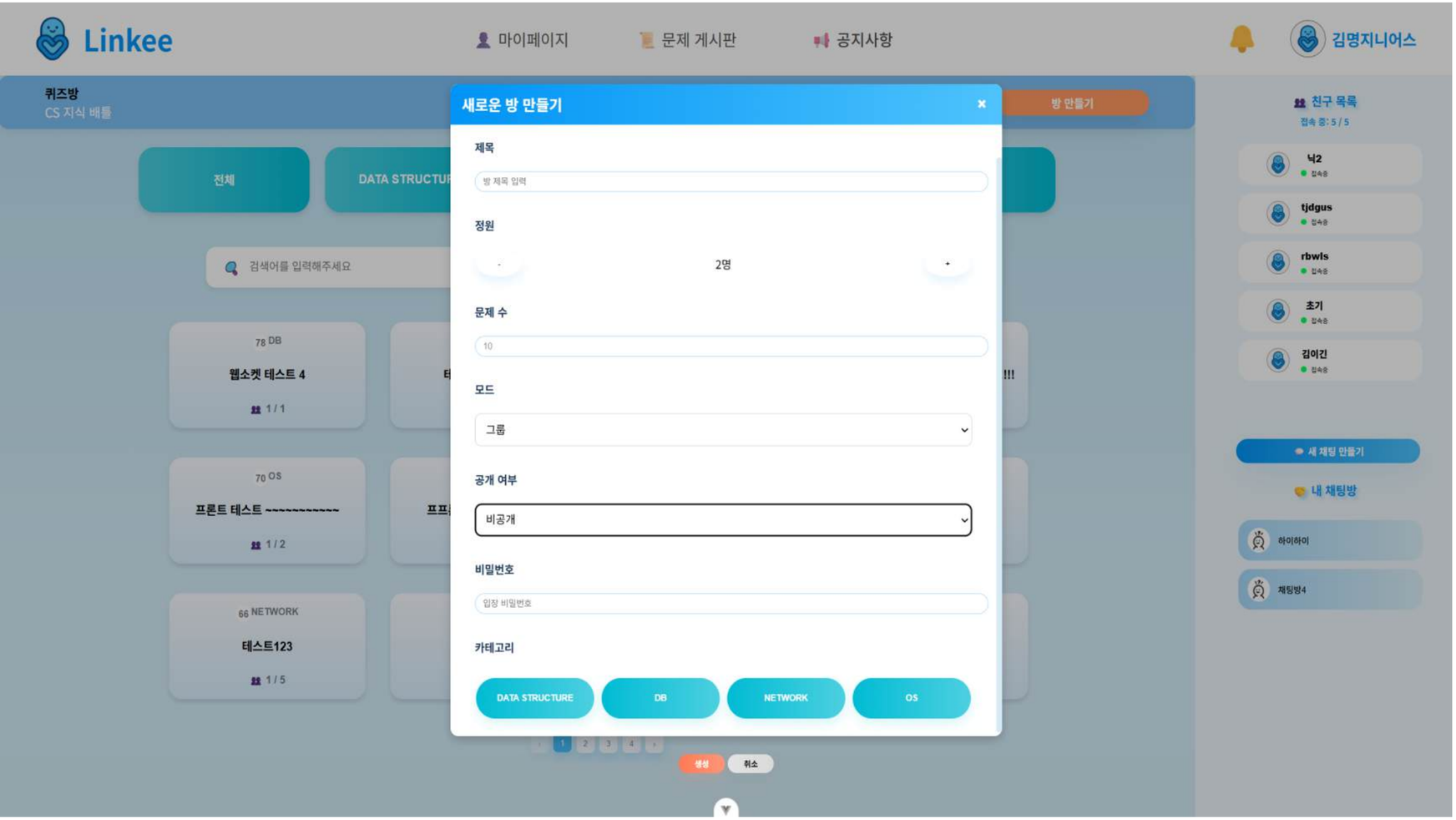
Location : '/quiz/rooms'

퀴즈방 목록 조회

현재 모집중인 퀴즈방 목록을
조회할 수 있다.

키워드 검색, 카테고리 분류,
페이징 처리를 통해 원하는 퀴즈방을
빠르고 정확하게 탐색할 수 있다.

quiz

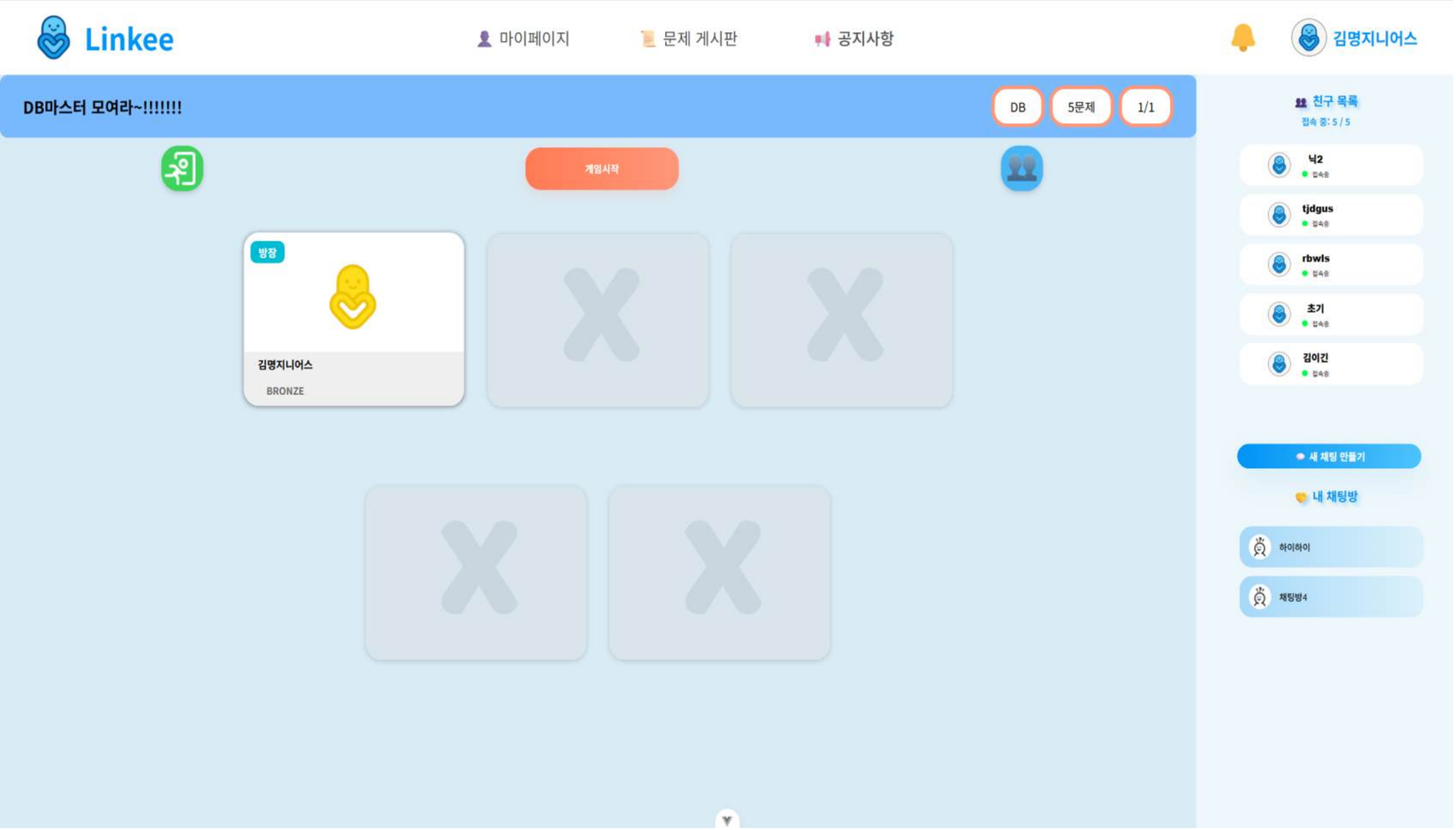


Location : '/quiz/rooms'

퀴즈방 생성

제목, 정원, 문제 수, 모드(싱글, 그룹),
공개여부(비공개 선택시 비밀번호 입력),
카테고리를 입력하고 방을 생성할 수 있다.

quiz



Location : '/quiz/rooms/:quizroomId/wait'

퀴즈방 대기실

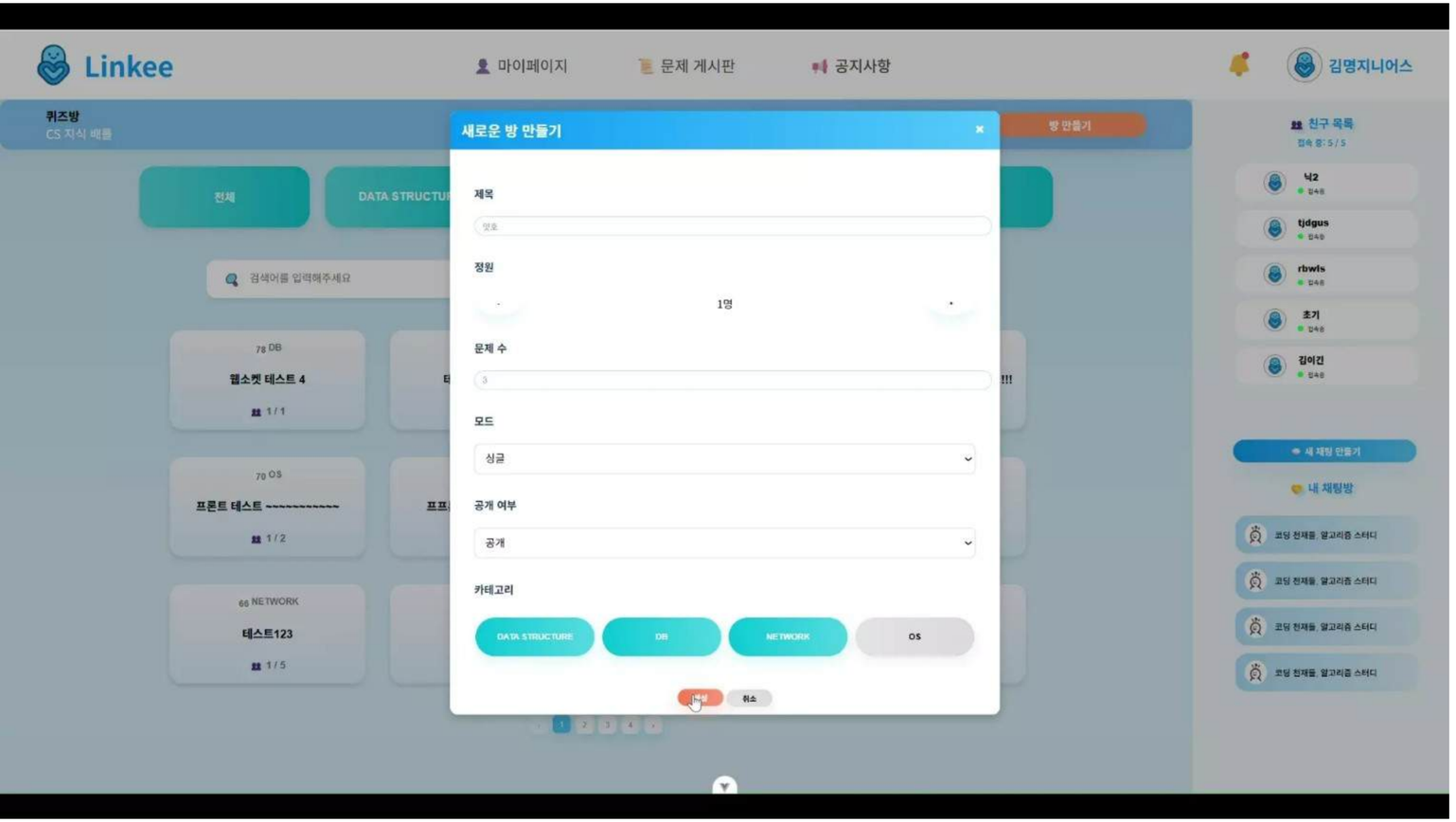
현재 방의 카테고리, 문제 수, 참여 인원 정보를 실시간으로 확인할 수 있다.

방장은 게임 시작 버튼을 통해 퀴즈를 시작할 수 있고, 참여자는 준비 상태를 표시할 수 있다.

라우터를 적용하여, 나가기 버튼 클릭 시 퀴즈방 목록 화면으로 이동하도록 구성

quiz

Location : '/quiz/rooms/:quizroomId/start'



퀴즈방

방장이 게임 시작 버튼을 누르면,
같은 퀴즈방에 있는 모든 플레이어에게
동일한 문제가 실시간으로 출력

플레이어는 문제를 보고 정답을 선택하여 제출할 수 있고,제출 여부는 실시간으로 공유

문제 종료 후에는정답이 공개되고,
다음 문제로 자동 진행

마지막 문제까지 종료되면 최종 순위 화면을 통해 게임 결과를 확인할 수 있다.

주요성과

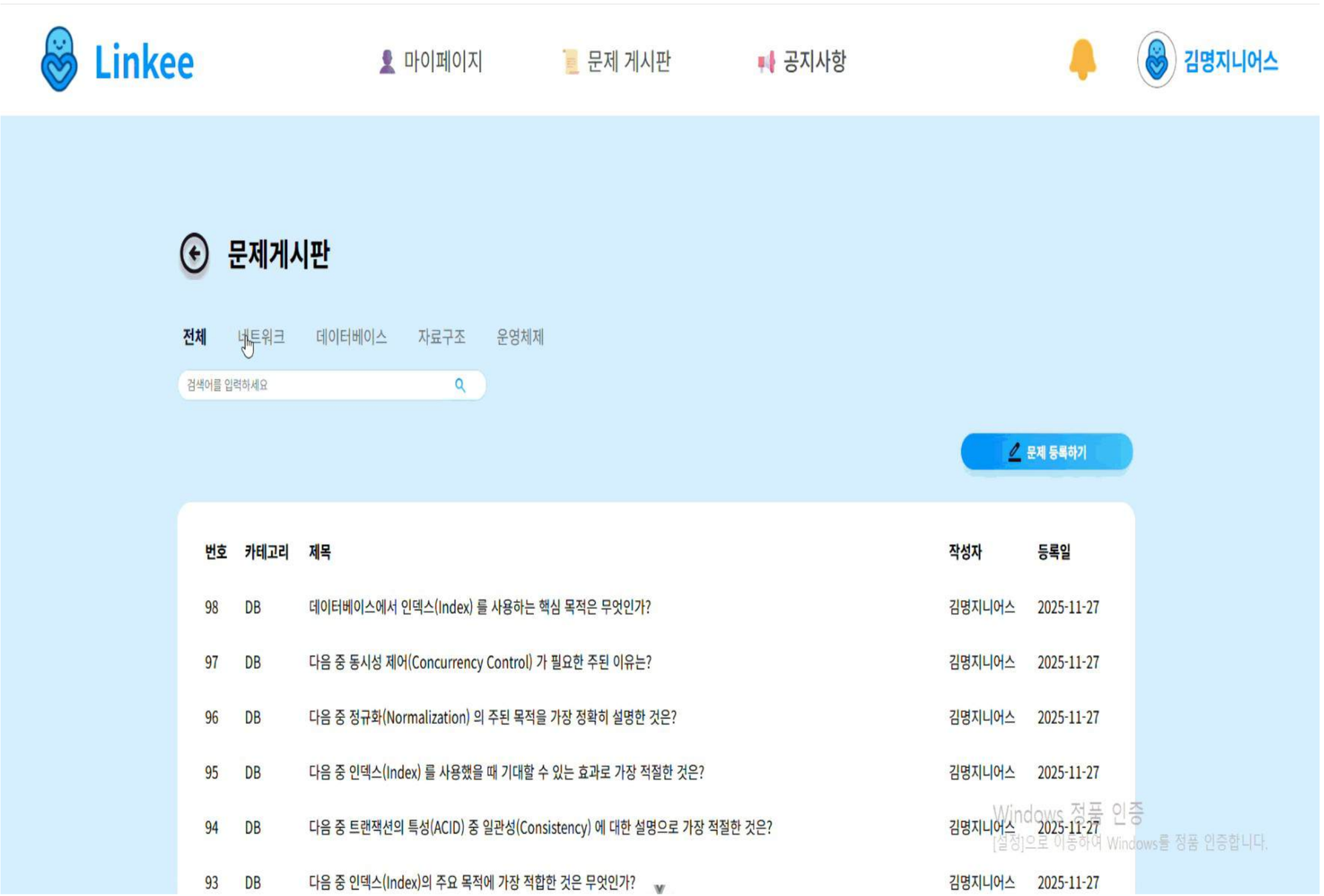
◎ 기본 레이아웃 header 와 aside 를 생성해 재사용성을 높임

◎ Base 버튼 , 인풋 , 검색 , 페이징 등 기본 컴포넌트를 만들어서
재사용성을 높이고 페이지 전체적인 안정감을 부여

◎ 각 기능별로 api 를 나누어서 유지보수와 확장성을 높임

◎ 피니아 스토어를 통해
유저정보 , 토큰값 , 채팅방 , stomp 웹소켓을 저장,
실시간으로 여러 컴포넌트에서 공유되고, 상태 변화가 자동으로 UI에 반영
채팅 상태와 WebSocket 연결을 하나로 공유

문제게시판



Question

Location : '/problems'

링키 문제 게시판

사용자가 작성한 문제 목록을 조회할 수 있다.

카테고리별 키워드별 검색을 통해
원하는 카테고리의
문제를 조회할 수 있다

사용자는 자유롭게 양식에 맞춰
CS 관련 문제를 등록할 수 있다.

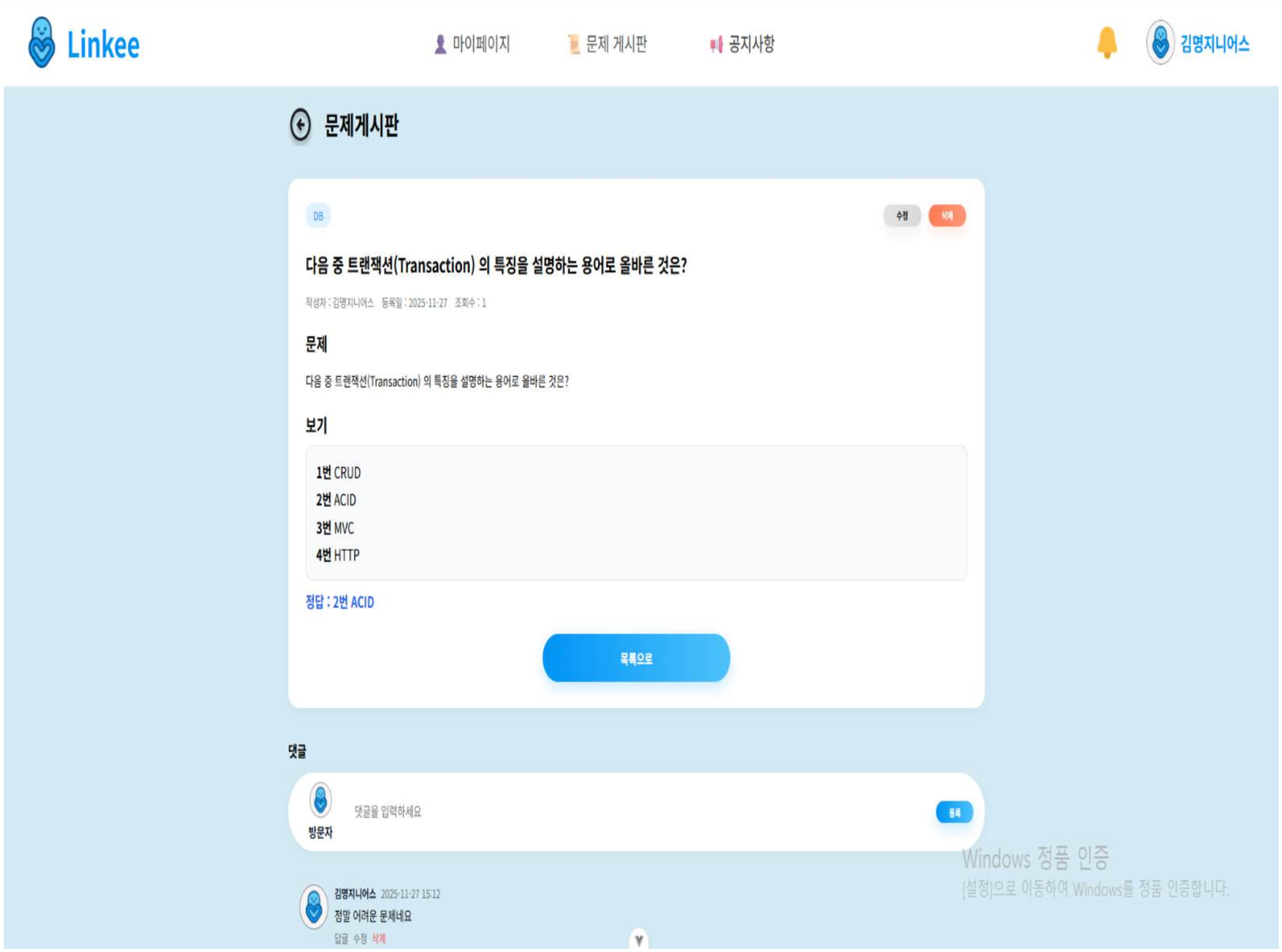


Question 등록

Location : '/problems/create'

문제 게시판 문제 등록

사용자는 자유롭게 양식에 맞춰 카테고리별 CS 관련 문제를 등록할 수 있다.



Question 조회/댓글

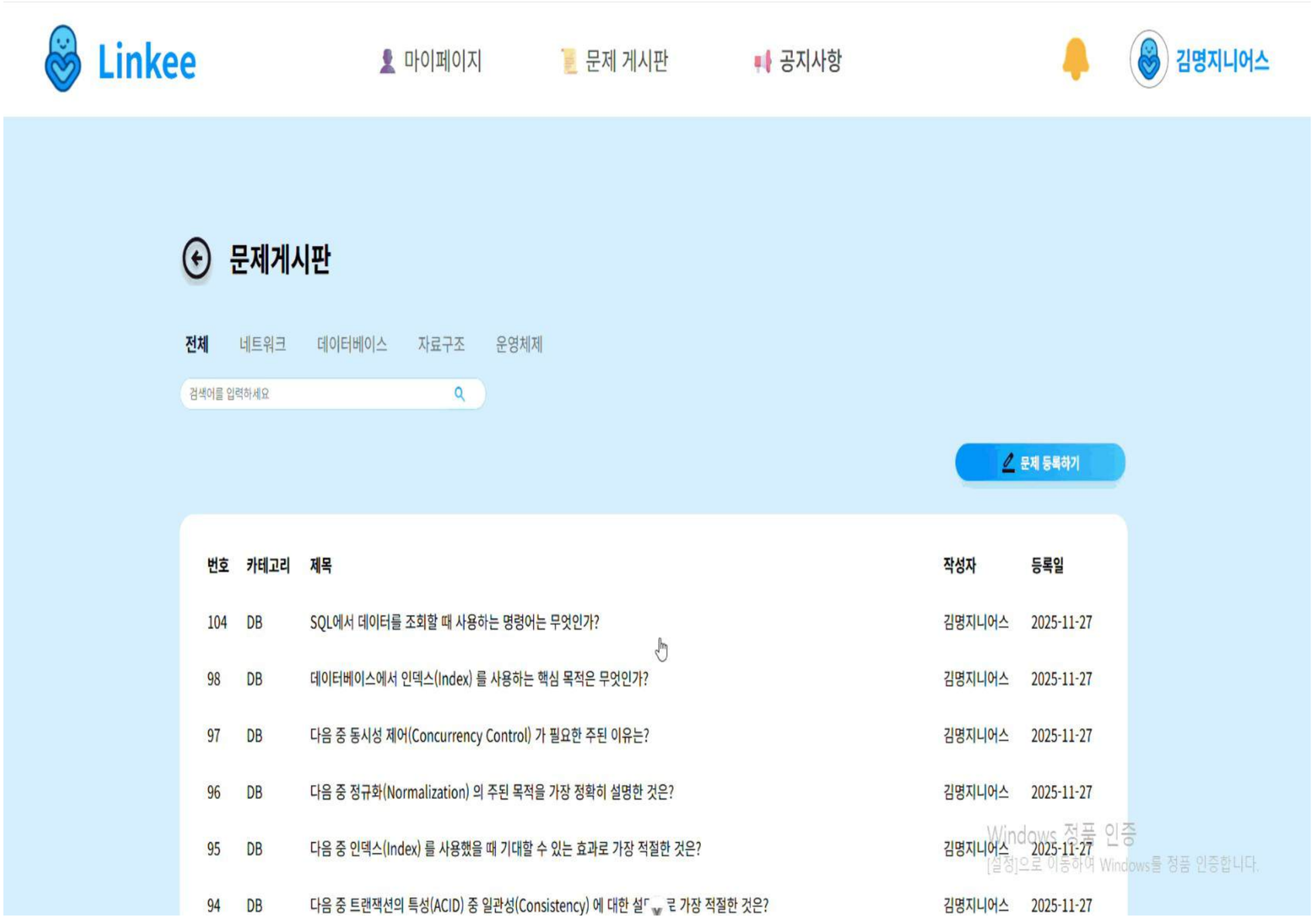
Location : '/problems/:problemId'

문제 게시판 문제 조회

사용자는 문제 목록에서 선택하여 문제 상세 내용을 조회할 수 있다.

본인이 작성한 게시글/댓글은 수정 삭제가 가능하다.

사용자는 상세 조회한 게시글에서 댓글을 등록/수정/삭제 할 수 있다.



Question 수정

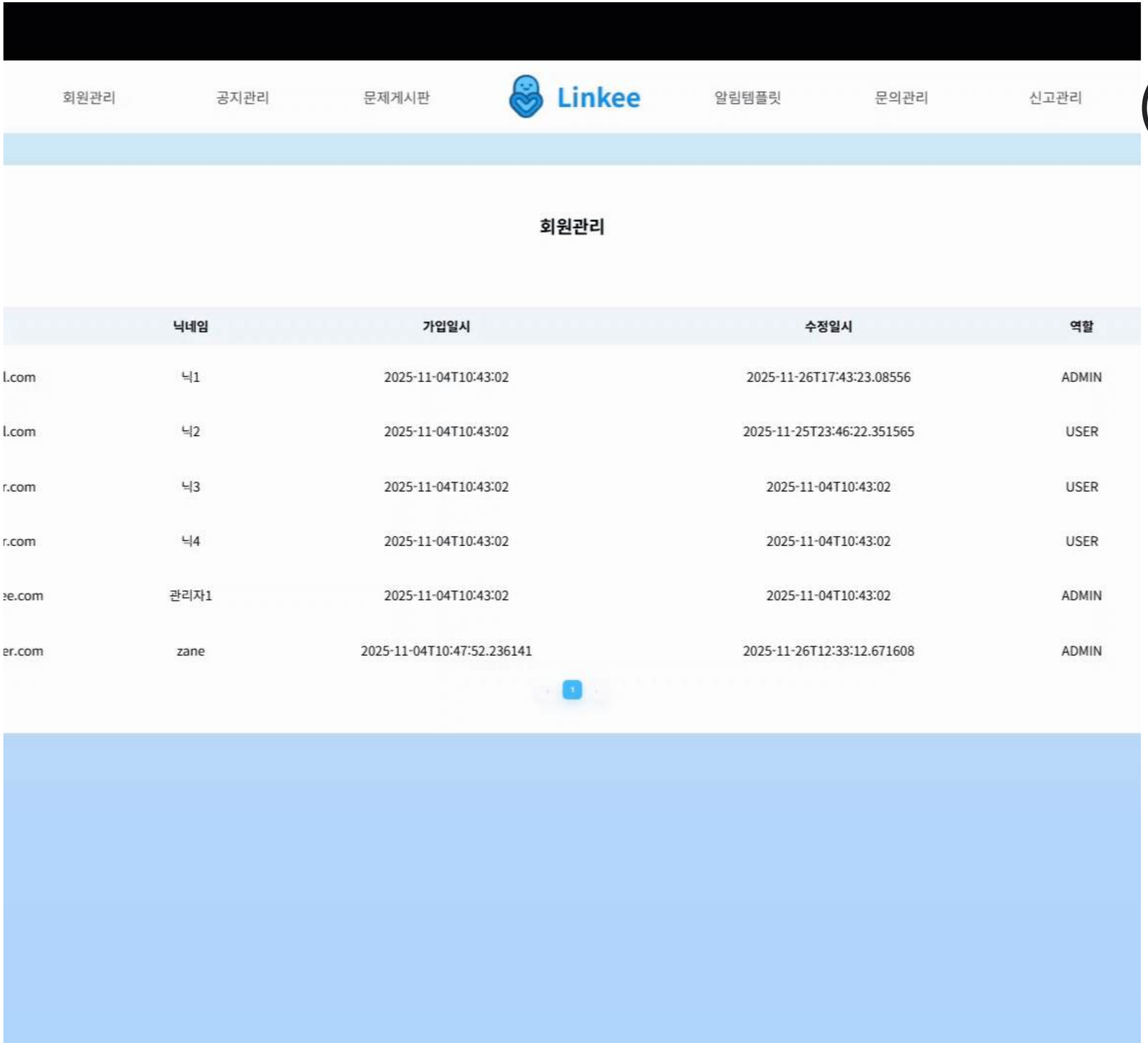
Location : '/problems/:problemId/edit'

문제 게시판 문제 수정

사용자는 본인이 작성한 문제 게시글을 수정할 수 있다.

이미 검증이 완료 된 게시물은 수정이 불가능하다.

관리자 페이지

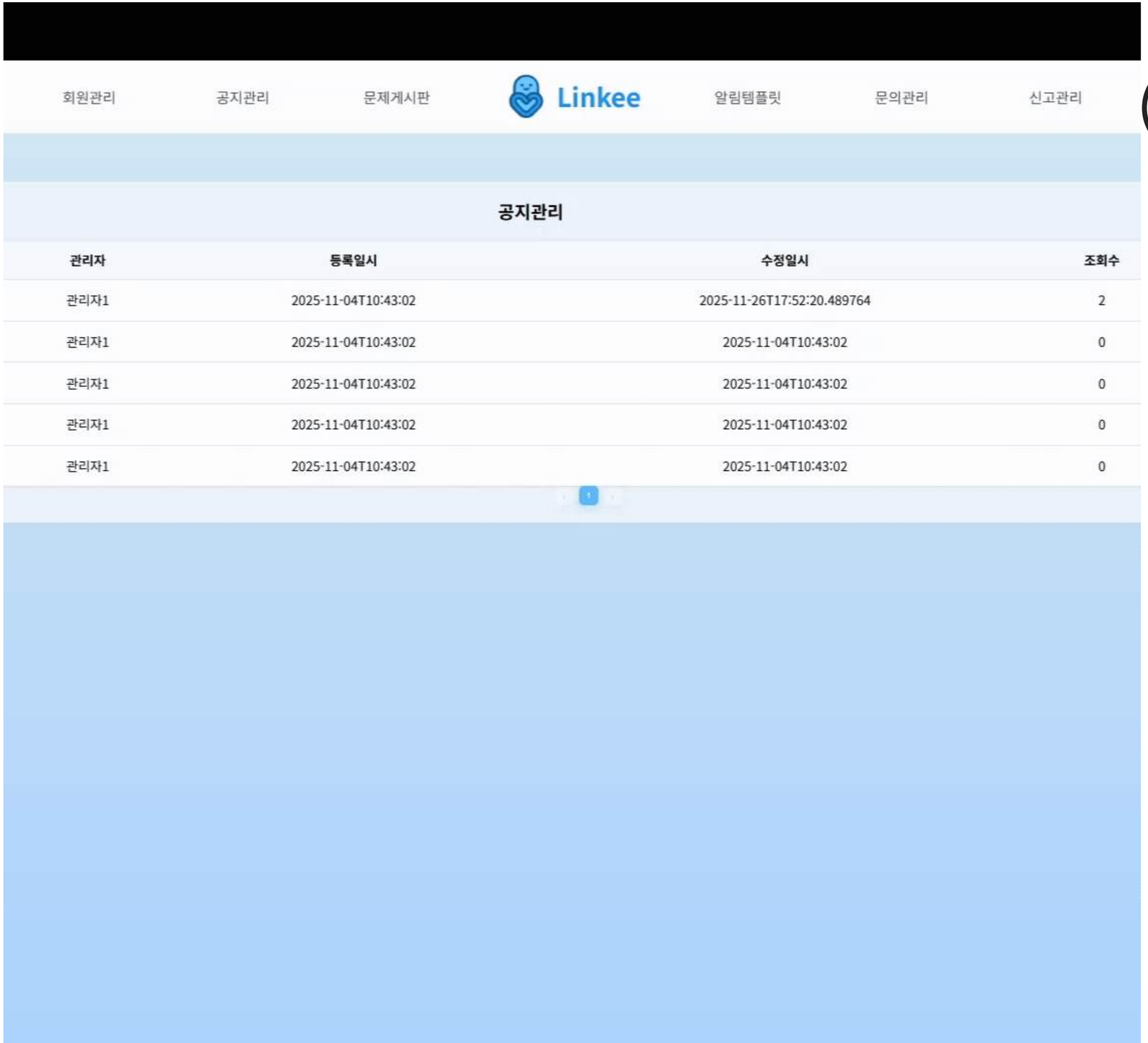


회원관리

Location : '/admin/users'

회원 목록 관리

관리자는 Linkee 서비스에 회원가입한 사용자들을 한 곳에서 관리할 수 있습니다. 해당 페이지에서는 사용자 권한 및 활성화 비활성화 여부를 설정합니다.



공지관리

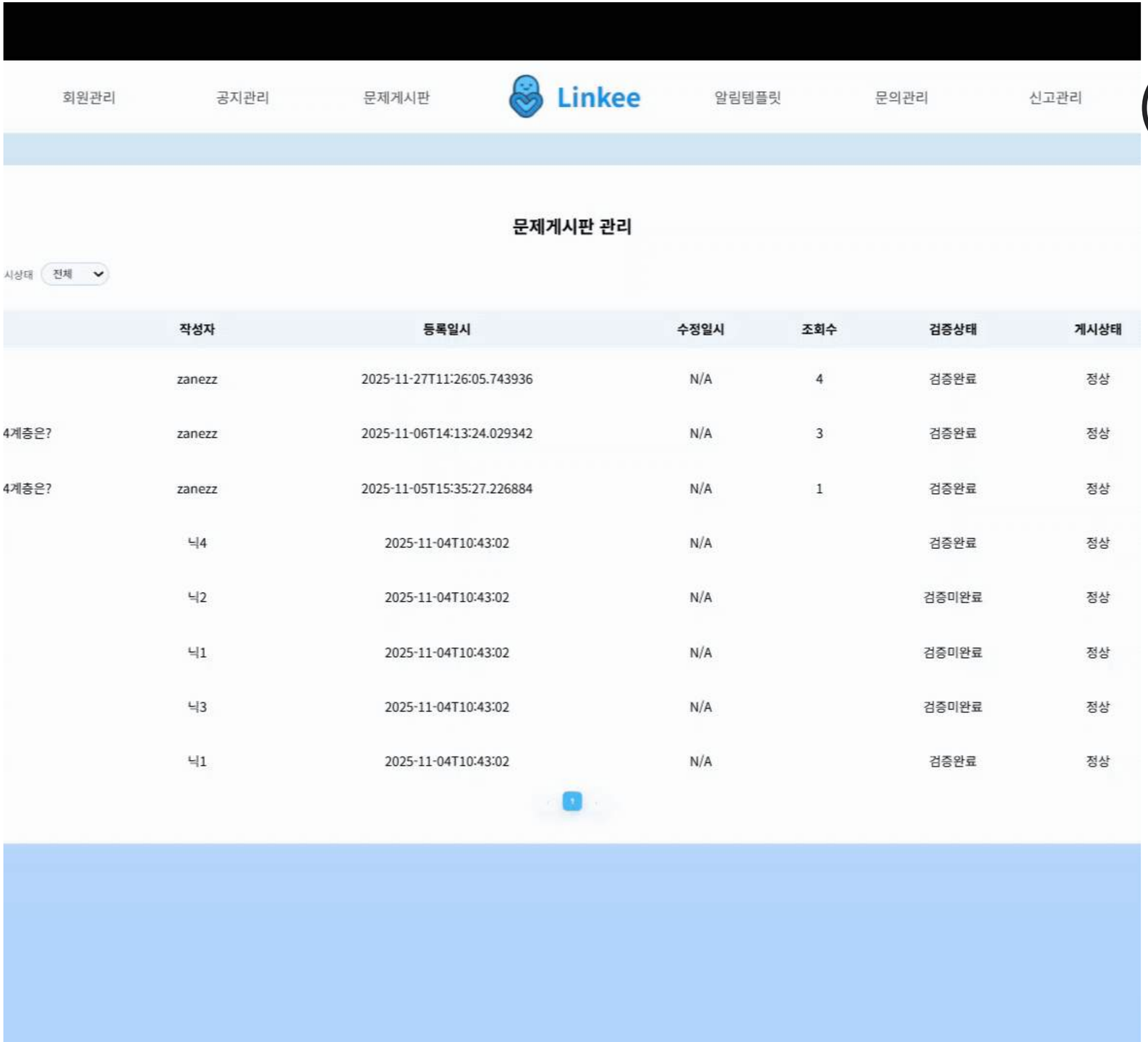
Location : '/admin/notices'
관리자 계정만 접근 가능한 공지사항 관리 화면을 별도로 구성하고, Pinia의 userRole로 접근 권한을 제어

등록 · 수정은 동일한 공지 작성 폼을 재활용하며, 수정 시에는 기존 데이터를 API로 가져와 입력 폼에 자동 바인딩

등록/수정 버튼 클릭 시 폼 데이터를 Axios API로 전송하고 성공 후에는 목록 페이지로 redirect

삭제는 공지 카드 또는 상세 화면에서 Confirm 모달을 띄운 뒤 해당 공지 ID로 DELETE API 요청을 보냄

모든 동작 성공 시에는 toast 메시지로 사용자 피드백을 제공하고, 목록 화면은 즉시 새로고침 또는 재조회하여 최신 상태를 반영

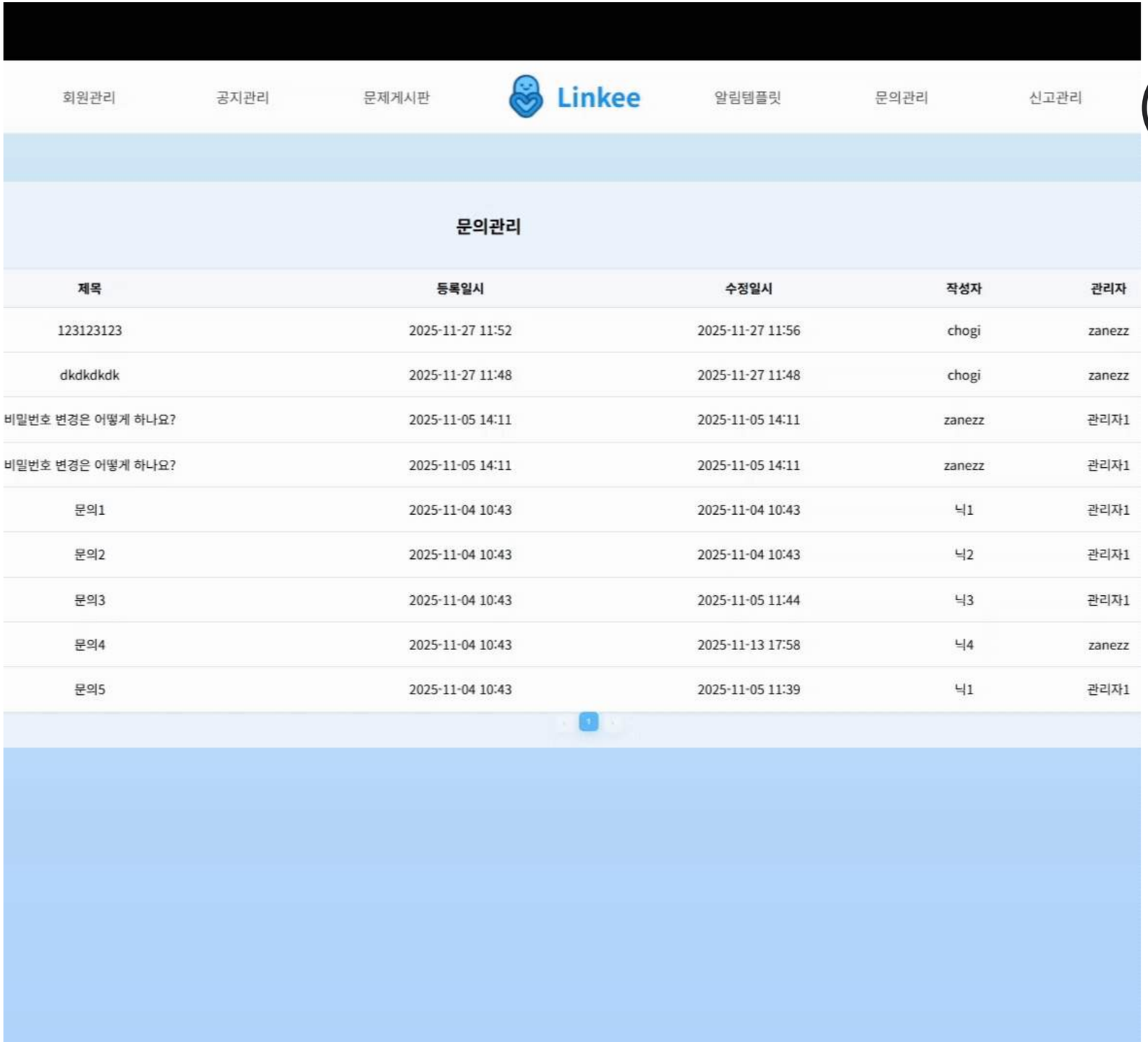


문제게시판 검증

Location : '/admin/problems'

문제게시판 검증

사용자가 올린 문제를 확인 후 적절한 문제라고 검증이 되면 관리자는 해당 페이지에서 문제를 검증 및 삭제할 수 있습니다.



문의관리

Location : '/admin/inquiries'

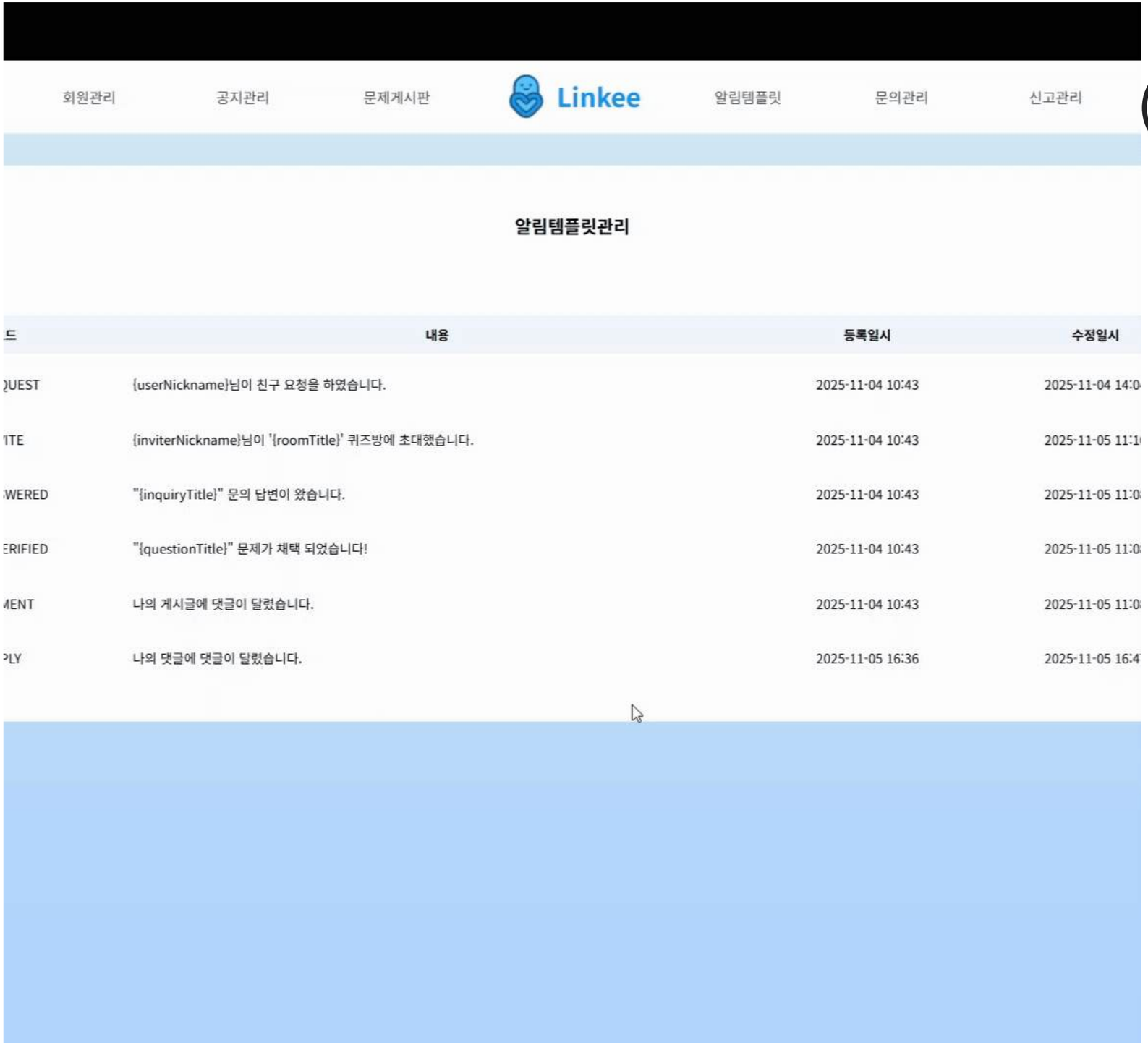
관리자 전용 문의 상세 페이지를 만들고, URL 파라미터로 전달된 inquiryId를 사용해 문의 상세 데이터를 API로 불러옴

문의 상세 화면에는 문의 내용과 함께 '답변 입력 폼(Textarea)을 추가해 관리자가 내용을 입력할 수 있도록 구성

"답변 등록" 버튼 클릭 시 Axios로 answerContent를 담아 PATCH/POST API 요청

등록 성공 시 상세 데이터가 즉시 갱신되도록 다시 상세 조회 API를 호출하거나 상태를 직접 업데이트

답변 완료된 문의는 answerStatus를 기반으로 상태 표시(UI 변경) 하고, 완료 안내 Toast를 출력한 뒤 목록 화면에서도 상태가 반영되도록 재조회

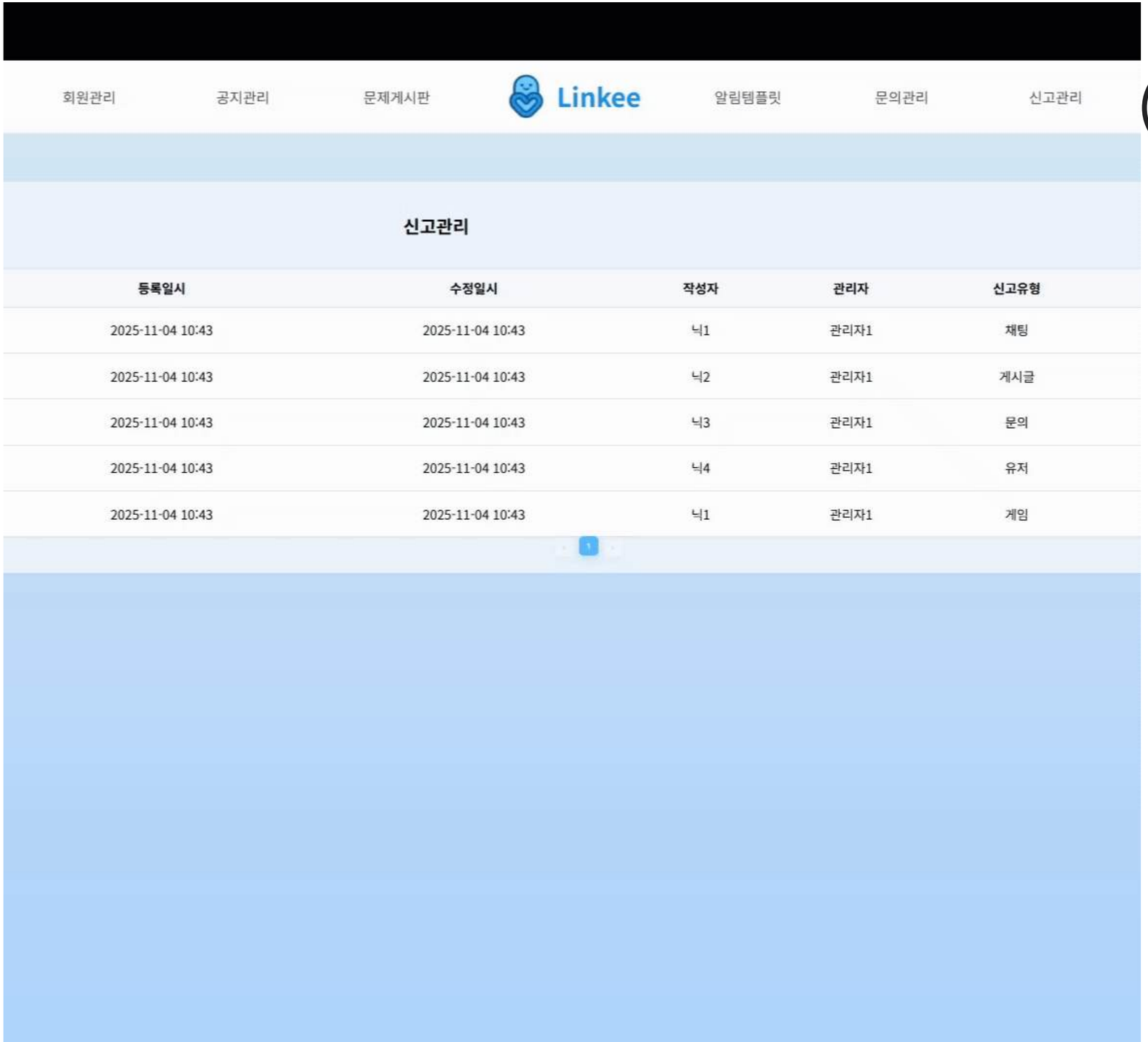


알림템플릿

Location : '/admin/templates'

알림템플릿 생성 및 수정

관리자는 사용자에게 전하는 알림의 내용을 생성, 수정, 삭제를 할 수 있습니다.



신고관리

Location : '/admin/reports'

관리자 전용 신고 관리 페이지를 구성하고, 초기 진입 시 Axios를 통해 신고 목록 조회 API를 호출하여 테이블(또는 카드 리스트)에 데이터를 렌더링

각 신고 항목에는 신고자 · 피신고자 · 사유 · 신고일자 · 상태가 표시되도록 UI를 구성하고, 상태별로 색상 · 배지를 사용해 가시성을 높임

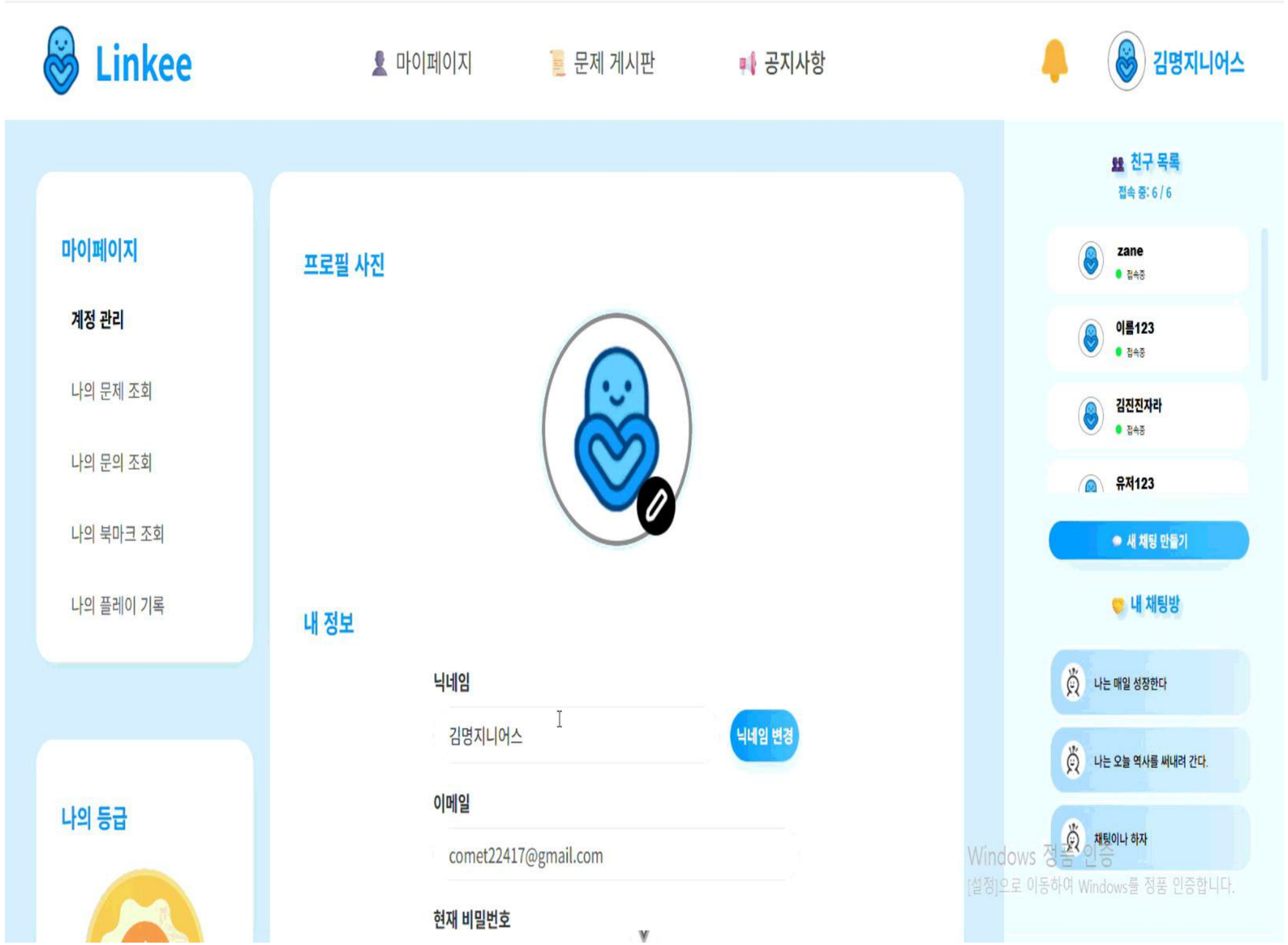
신고 항목 선택 시 상세 정보를 보여주는 신고 상세 모달 or 별도 상세 페이지를 연결해 신고 대상의 정보와 신고 내용을 확인할 수 있게 함

관리자는 상세 화면에서 처리, 경고, 반려 등의 조치를 선택할 수 있으며, 버튼 클릭 시 Axios로 신고 상태 변경 API를 호출할 수 있음

상태 변경이 성공하면 Toast로 알림을 띄우고, 신고 목록을 즉시 재조회해 리스트 화면에도 새로운 상태가 반영되도록 함

마이 페이지

계정 관리



Location : '/mypage/profile'

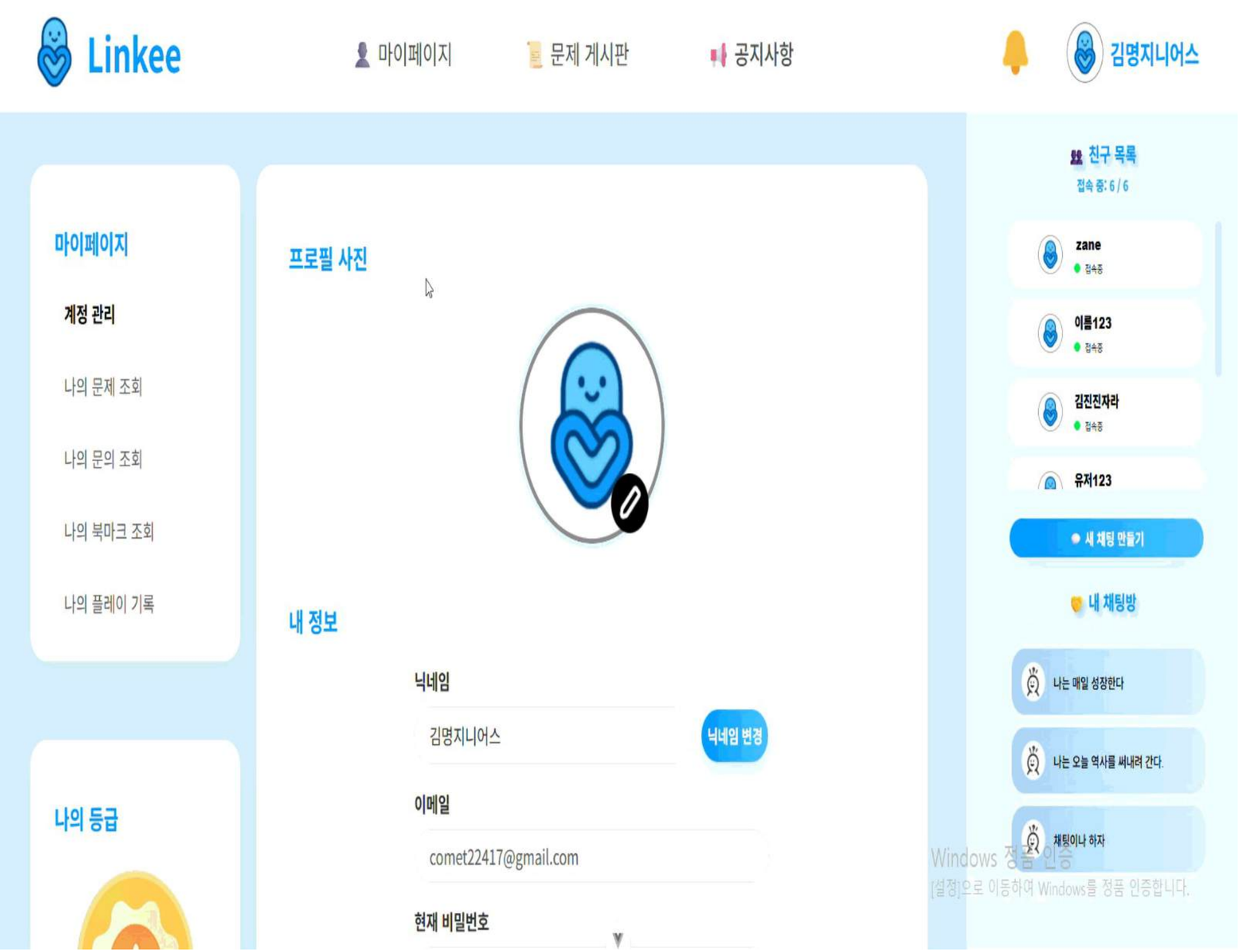
상단바에서 마이페이지 버튼을 누르면 가장 먼저 계정 관리쪽으로 들어오게 됩니다.

아직 기능은 넣지 않았지만 프로필 사진을 변경할 수 있게 할 예정이며

하단에 '내 정보'에서는 엘리멘탈 폼 방식을 이용하여 각 필드를 “개별 검증”하게 되어있습니다. 닉네임 수정 후 오른쪽에 '닉네임 변경' 누르면 변경

현재 비밀번호 입력 후 새 비밀번호, 비밀번호 확인 후 문제가 없으면 수정이 되며 문제가 발생되면 검색 바 하단에 빨간 글씨로 메시지가 발생합니다.

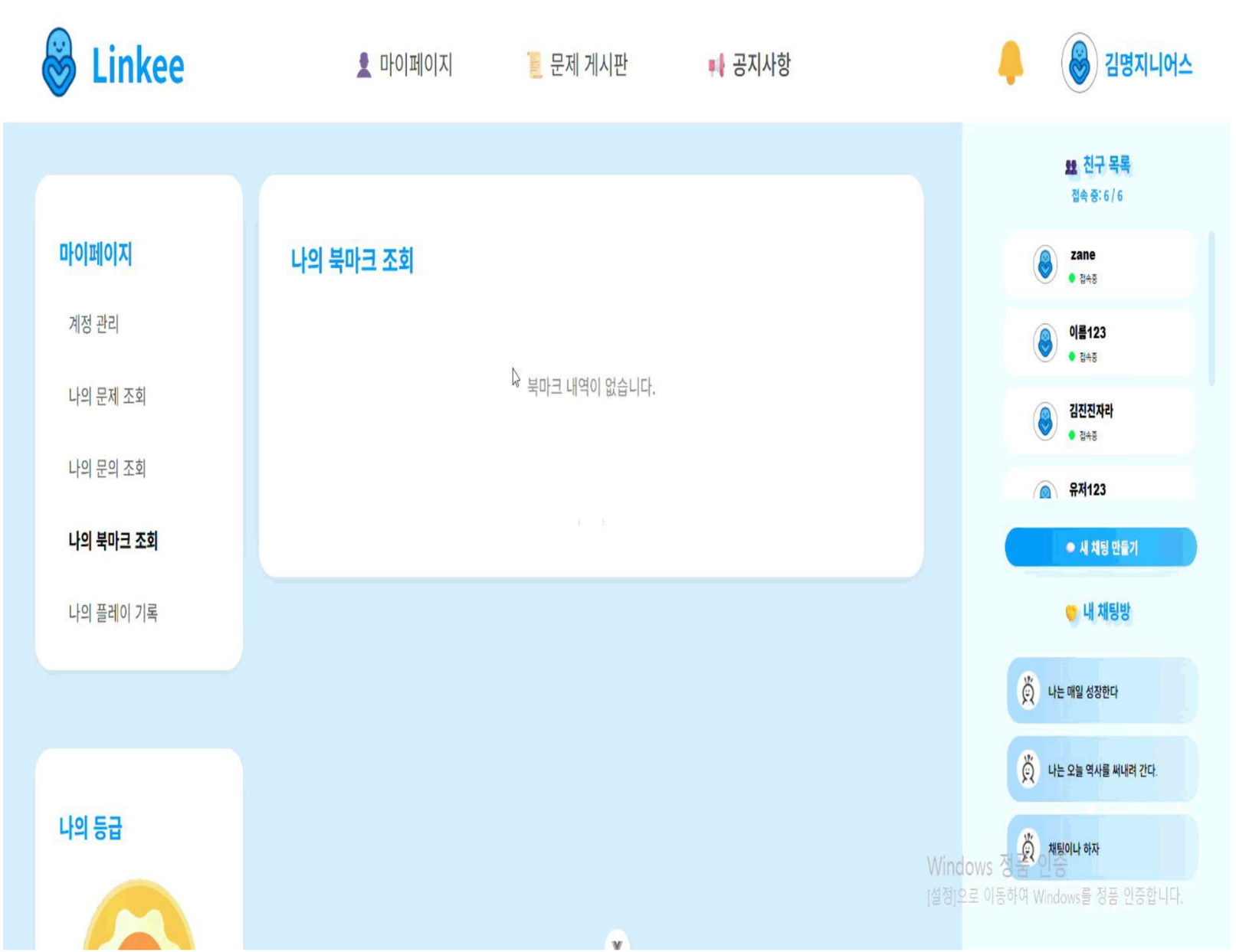
회원탈퇴 시 해당 계정은 비활성화 처리가 되며 로그인 페이지로 redirect 됩니다.



나의 문제 조회

Location : '/mypage/questions'

사용자는 왼쪽의 탭을 선택하여 내가 작성한 문제 목록을 조회, 상세 조회할 수 있다.



나의 북마크 조회

Location : '/mypage/bookmark'

사용자는 왼쪽의 탭을 선택하여 북마크 목록을 조회/ 상세조회 할 수 있다.

트러블슈팅

문제 상황

프론트엔드 개발 중 **로그인하지 않은 상태에서 알림 API를 호출**하면서 다음과 같은 문제가 발생했다.

- CORS 오류
- Network Error (AxiosError: Failed to fetch alarm boxes)
- Naver OAuth 로그인 URL로 강제 리다이렉트됨

문제 원인

`UserNavbar.vue`가 전역 컴포넌트 이 컴포넌트는 로그인 페이지를 포함한 거의 모든 페이지에서 렌더링된다.
즉, 로그인 여부와 상관없이 화면에 나타난다.

해결 방법

authStore.js에서 로그인 상태 확인을 위해 해당 부분 작성 -> const isLoggedIn = computed(() => !!accessToken.value && !!user.value);
✔ isLoggedIn을 이용해 로그인 여부를 체크할 수 있다.

```
onMounted(() => {
  if (auth.isLoggedIn) {
    fetchNotifications();
  }
});
```

onMounted 라이프사이클 훅 안에서 로그인 여부를 확인 후 알림 메소드를 실행

트러블슈팅2

친구 초대 후 DB에는 저장되지만 프론트 UI에는 즉시 반영되지 않는 문제

증상

친구 초대 API(POST /chat/rooms/{roomId}/invite)는 성공적으로 200 OK를 반환하고,
DB에도 새로운 초대 멤버가 정상 저장됨.

그러나 프론트 Vue 화면에서는 채팅방 멤버 목록이 즉시 갱신되지 않고,
이전 상태의 멤버만 그대로 출력됨.

특히 초대한 직후 열려 있는 ChattingModal에서는 새 멤버가 UI에 즉시 반영되지 않음.

원인 분석

* 초대 성공 시 UI에서는 **기존 room.members 배열만 참조**하고 있었고
초대 후 **멤버 목록을 새로 요청(fetch)**하지 않음.
반응형 저장 방식 (`rooms.value.find(...)`)도 불안정하여 즉시 갱신이 되지 않음.

해결 방법

초대 완료 후 `fetchRoomMembers(roomId)` API 재호출하여 최신 멤버 목록으로 갱신.
`rooms.value`에서 해당 room 객체를 찾아 직접 members를 교체하는 방식으로 반응형 보장.
rooms 배열을 직접 변경하는 것이 아니라, 반응성을 유지하도록 전체 객체 재할당